

Laporan TBFO

IF2224 - Teori Bahasa Formal dan Otomata



Disusun Oleh:

Geraldo Artemius - 13524005
Mikhael Andrian Yonatan - 13524051
Junior Natra Situmorang - 13524055
Reynard Nathanael - 13524103
Nicholas Luis Chandra - 13524105

DAFTAR ISI

Laporan TBFO.....	1
DAFTAR ISI.....	2
DAFTAR TABEL.....	3
DAFTAR GAMBAR.....	4
BAB I.....	5
1.1. Semantic Analysis.....	5
1.1.1. Abstract Syntax Tree (AST).....	6
1.1.2. Decorated Abstract Syntax Tree (Decorated AST).....	7
1.1.3. Syntax-Directed Translation (SDT).....	8
1.1.4. L-Attributed Grammar dan Depth-First Search.....	8
1.2. Manajemen Symbol Table Terdistribusi.....	9
1.3. Aturan Kompatibilitas Tipe.....	9
BAB II.....	10
2.1. Struktur Direktori.....	10
2.2. Implementasi Program.....	12
2.3. Aturan Syntax-Directed Translation Scheme (SDT) pembentukan AST.....	58
2.4. Predefined Identifier.....	62
2.5. Type Compatibility Rule.....	63
BAB III.....	70
3.1 Test Case 1.....	70
3.2 Test Case 2.....	73
3.3 Test Case 3.....	76
3.4 Test Case 4.....	79
3.5 Test Case 5.....	82
BAB IV.....	85
4.1. Kesimpulan.....	85
4.2. Saran.....	85
BAB V.....	86
BAB VI.....	87

DAFTAR TABEL

Tabel 2.3 Aturan SDT pembentukan AST.....	62
Tabel 2.4 Predefined Identifier.....	63
Tabel 2.5.1 Tipe Dasar.....	64
Tabel 2.5.2 Structured Type.....	64
Tabel 2.5.3 Rule untuk Expression dan Statement.....	66
Tabel 2.5.4 Compatibility Rule.....	67
Tabel 2.5.5 Assignment Compatibility.....	69

DAFTAR GAMBAR

Gambar 1.1.1.1 Parse Tree dan Abstract Syntax Tree (AST).....	7
Gambar 1.1.1.2 Contoh Abstract Syntax Tree (AST)	7

BAB I

TEORI SINGKAT

1.1. Semantic Analysis

Semantic analysis merupakan tahap ketiga dalam compiler. Kata 'Semantic' dalam Semantic Analysis adalah istilah linguistik atau ilmu bahasa, Analisa ini bisa dalam bentuk bahasa atau makna bahasa. Dalam pemeriksaan Semantik yang diperiksa adalah makna dari setiap pemakaian token dari sumber program. Tahapan semantic analyzer merupakan jembatan antara fase analysis dan fase synthesis pada compiler. Pada tahapan ini akan dihasilkan suatu kode yang executable pada kompilasi yang sederhana lalu dimanipulasi dengan berbagai optimization dari translator sampai nanti benar-benar executable dihasilkan.

Pengecekan semantic yang dilakukan oleh **Semantic Analysis** terdiri dari:

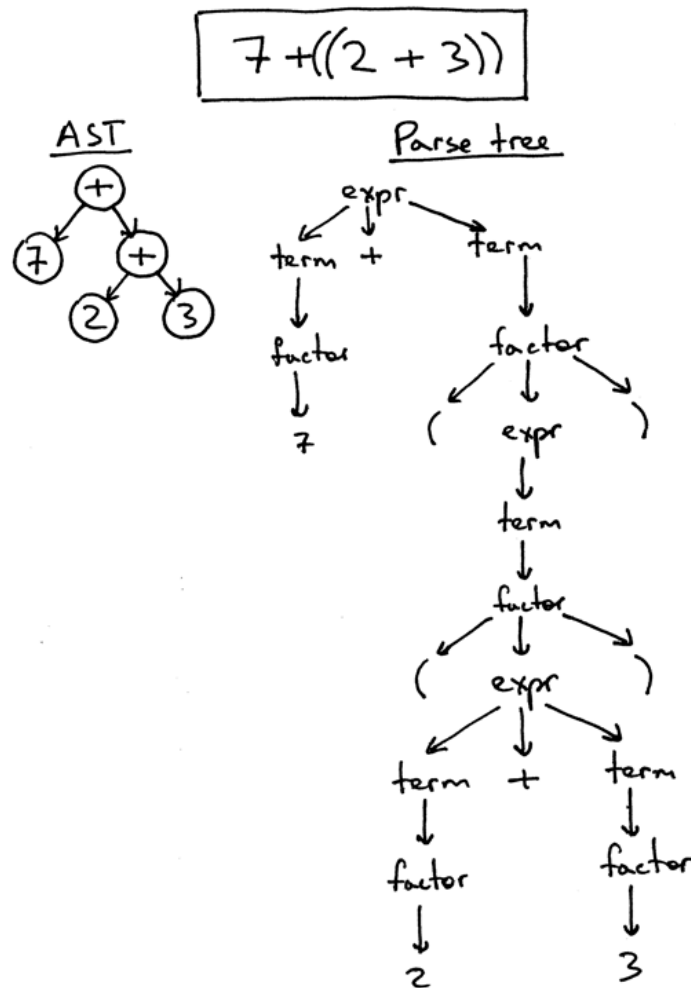
- ***Type Checking (Pengecekan Tipe)***
Memastikan operator dan operand memiliki tipe yang kompatibel.
contoh: tidak menjumlahkan string dengan integer, perbandingan char terhadap string.
- ***Symbol Table Management (Pengecekan Deklarasi)***
Memastikan setiap identifier (variabel/fungsi) telah dideklarasikan sebelum. digunakan dan tidak dideklarasikan ulang dalam cakupan yang sama.
- ***Scope Resolution (Pengecekan Lingkup)***
Menentukan validitas akses variabel berdasarkan hierarki blok kode.
Contoh: scope global vs scope local.
- ***Control Flow Validation (Pengecekan Alur Kontrol)***
Memastikan struktur alur kontrol yang logis.
Contoh: instruksi break/continue hanya dalam loop, fungsi dipastikan memiliki return pada semua cabang kontrol, warning untuk unreachable code.

Semantic Analysis akan melakukan anotasi terhadap parse tree yang telah dihasilkan oleh parser dan menghasilkan sebuah **decorated abstract syntax tree (AST)**. Anotasi berupa (dan tak terbatas kepada) penambahan informasi, seperti tipe data dan referensi simbol (identifier) ke symbol table.

1.1.1. Abstract Syntax Tree (AST)

Abstract Syntax Tree pada dasarnya adalah **bentuk ringkas dari parse tree**, dimana kita hanya akan menyimpan informasi yang diperlukan untuk tahapan selanjutnya dan membuang informasi yang sudah tidak lagi diperlukan.

Struktur dasar sebuah **parse tree**, didasarkan pada **aturan Grammar** untuk melakukan pengecekan syntax. Namun pada tahap **Semantic Analisis** dan selanjutnya, kita tidak lagi memerlukan konteks tersebut. Sehingga pada **AST**, kita berpindah fokus menuju **aturan semantic**, yaitu mengenai **hubungan antar operator dan operand**. Aturan tersebut akan menjadi penting untuk pengecekan semantic dan tahapan selanjutnya.

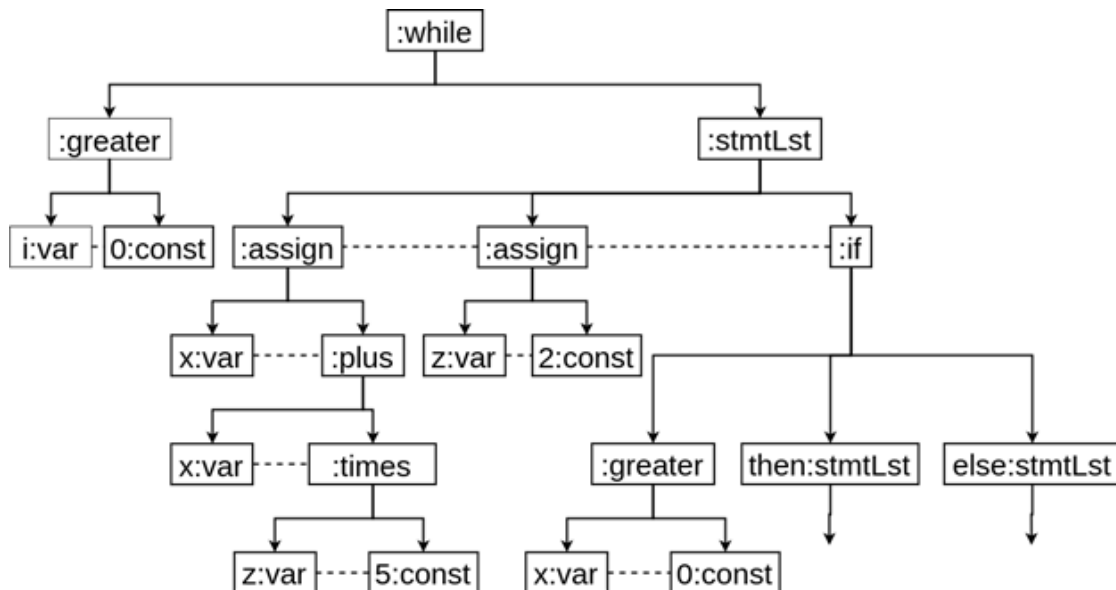


Gambar 1.1.1.1 Parse Tree dan Abstract Syntax Tree (AST) untuk ekspresi matematika
(Sumber: <https://ruslanspivak.com/lsbasi-part7>)

Sebagai ilustrasi, untuk potongan kode berikut.

```
/* TIDAK DIPASTIKAN SESUAI BAHASA ARION */
while (i > 0) {
    x = x + z * 5;
    z = 2;
    if (x > 0) then { ... } else { ... }
}
```

Kode tersebut akan menghasilkan Abstract Syntax Tree:



Gambar 1.1.1.2 Contoh Abstract Syntax Tree (AST)
(Sumber: <https://nus-cs3203.github.io>)

1.1.2. Decorated Abstract Syntax Tree (Decorated AST)

Semantic Analysis akan melakukan **anotasi** terhadap AST hasil konversi dari parse tree untuk menghasilkan sebuah **Decorated AST**. Anotasi yang dilakukan bertujuan memberikan konteks untuk **pengecekan semantic**, seperti penambahan simbol table, tipe data, penambahan reserve keywords, pelacakan identifier, penanda tingkatan scope, dan sebagainya. Implementasi Decorated AST memang beragam, namun dalam tugas ini, berbasis **3 symbol table**, yaitu **tab**, **btab**, dan **atab**. Jadi Decorated AST merupakan hasil dari proses pengecekan semantic pada Semantic Analysis.

Berikut adalah beberapa istilah yang perlu dipahami:

1. Attributed Grammar

Grammar yang dilengkapi dengan atribut dan aturan semantik.

2. Symbol Table

Struktur data yang minimal menyimpan informasi nama identifier, tipe datanya dan scopenya. **Symbol table menggunakan struktur data stack.**

3. Visitor / Visit Functions

Untuk setiap jenis node akan memiliki fungsi `visit_<node>` yang akan:

- 1) Memanggil visit pada child-nya
- 2) Mengakses atau memperbarui symbol table
- 3) Menghitung atribut/tipe semantik
- 4) Menambahkan anotasi pada node

1.1.3. Syntax-Directed Translation (SDT)

Syntax-Directed Translation (SDT) adalah metode yang memanfaatkan struktur sintaks (aturan produksi grammar) untuk mengarahkan proses terjemahan atau analisis semantik. Pada tahap Semantic Analysis, *Syntax-Directed Translation Scheme* dimanfaatkan untuk menyisipkan aksi semantik (*semantic action*) pada aturan produksi grammar untuk mengkonversi Parse Tree menjadi *Abstract Syntax Tree* (AST). Aksi semantik ini berfungsi untuk mengevaluasi nilai atribut dari setiap node yang dikunjungi. Selama proses translasi ini, informasi sintaksis yang sudah tidak diperlukan (seperti tanda kurung berlebih atau *keyword* seperti BEGIN, END, SEMICOLON) akan dihilangkan. Node-node tersebut kemudian digabungkan dari bawah ke atas (*bottom-up*) untuk dirakit menjadi satu kesatuan AST yang ringkas dan utuh

1.1.4. L-Attributed Grammar dan Depth-First Search

Dalam penelusuran Semantic Analysis, digunakan pendekatan algoritma *L-Attributed Grammar*. Proses dekorasi AST (termasuk *Type Checking* dan *Scope Checking*) dilakukan dengan menelusuri AST secara *Depth-First Search* (DFS) menggunakan fungsi *visitor*. Algoritma ini memungkinkan compiler untuk memproses informasi atau atribut semantik dari kiri ke kanan secara berurutan. Sepanjang penelusuran DFS ini, sistem juga mengelola *scope* blok program dengan melakukan *Push* pada *stack Symbol Table* saat memasuki blok baru (seperti prosedur atau fungsi) dan melakukan *Pop* saat keluar dari blok tersebut.

1.2. Manajemen Symbol Table Terdistribusi

Untuk menjaga konsistensi tipe data dan validitas deklarasi, Semantic Analyzer mengandalkan *Symbol Table*. Dalam arsitektur bahasa yang kompleks, struktur data ini tidak hanya berupa satu tabel tunggal, melainkan dipecah menjadi beberapa tabel spesifik agar pencarian lebih terorganisir. Dalam kompilator ini, *Symbol Table* menggunakan 3 struktur tabel utama:

1. **tab (Identifier Table):** Tabel utama yang digunakan untuk menyimpan detail informasi mengenai *identifier* (nama variabel, konstanta, tipe data, prosedur, fungsi). Tabel ini juga mencatat *lexical level (lev)* atau tingkatan *scope* dari sebuah *identifier*.
2. **btab (Block Table):** Tabel ini secara khusus digunakan untuk merepresentasikan dan menyimpan informasi mengenai blok (seperti blok prosedur, blok fungsi, atau definisi tipe *record*).
3. **atab (Array Table):** Tabel yang menyimpan karakteristik komposit dari tipe data *array*, termasuk tipe indeks, batas bawah indeks (*low*), batas atas indeks (*high*), tipe elemen (*etyp*), serta ukuran per elemen dan total memori *array* tersebut.

1.3. Aturan Kompatibilitas Tipe

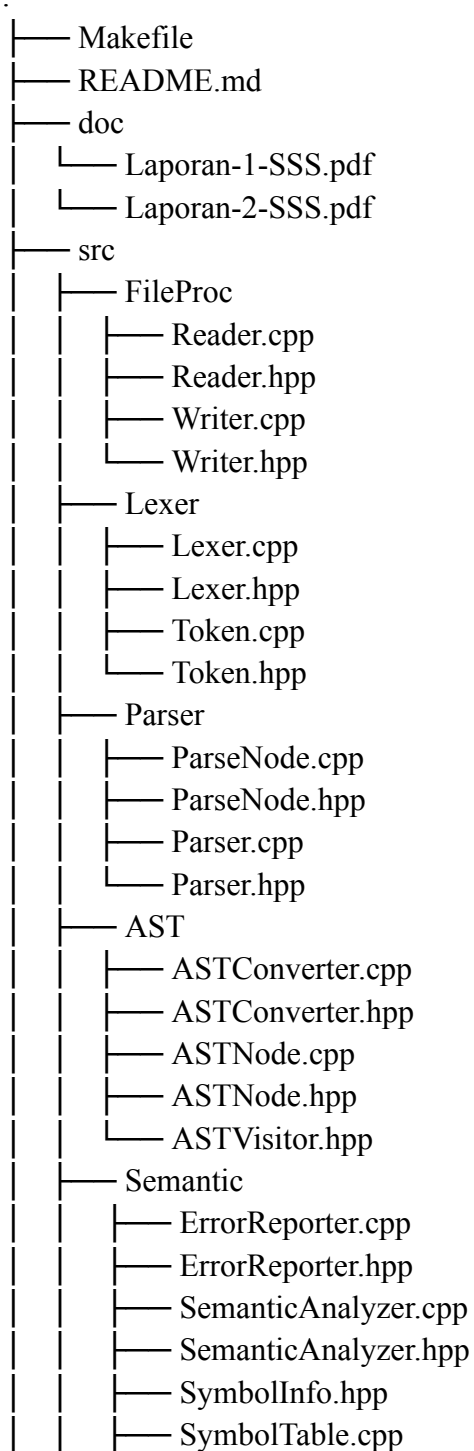
Sebuah "type" di dalam bahasa pemrograman menentukan "bentuk data" yang bisa ditampung oleh variabel, ekspresi, maupun kembalian fungsi. *Type Compatibility Rule* adalah himpunan aturan semantik yang mendikte bagaimana tipe-tipe data ini dapat berinteraksi secara sah. Terdapat dua tingkatan kompatibilitas tipe yang divalidasi:

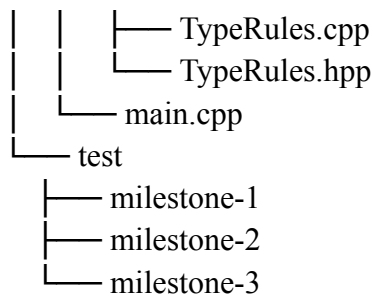
1. **Compatible Types:** Dua tipe data disebut compatible (kompatibel) jika memenuhi syarat kelogisan, seperti keduanya adalah tipe yang sama persis, satu tipe adalah subrange dari tipe yang lain (contoh: [1..10] kompatibel dengan Integer), atau keduanya merupakan string dengan panjang yang seragam.
2. **Assignment-Compatible:** Ini merupakan regulasi lebih spesifik tentang boleh tidaknya suatu nilai ekspresi dimasukkan (di-assign) ke suatu variabel tujuan. Tipe akan dianggap assignment-compatible contohnya jika variabel penerima bertipe Real sedangkan nilai yang dieksekusi adalah Integer, atau ketika value ekspresi dijamin merupakan subset dari range yang diizinkan pada variabel tujuannya. Jika aturan ini dilanggar (misalnya memberikan nilai 11 ke variabel berindeks 1..10), Semantic Analyzer harus menolak operasi tersebut (Assignment-Incompatible Error)

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Struktur Direktori





Berikut struktur folder kami, masih sama dengan laporan sebelumnya, secara garis besar :

- Folder doc berisi dokumen laporan Tugas Besar dalam bentuk dokumen PDF.
- Folder src berisi *source code* program dalam bahasa C++ yang mencakup implementasi mesin DFA (Lexer.cpp, Lexer.hpp) pada folder Lexer, definisi token (Token.cpp, Token.hpp) pada folder Lexer juga, pemrosesan file (Reader.cpp, Reader.hpp, Writer.cpp, Writer.hpp) pada folder FileProc, integrasi seluruh modul (main.cpp), implementasi modul *parse tree* pada folder Parser (ParseNode.hpp, ParseNode.cpp, Parser.cpp, dan Parser.hpp). Selain itu, saat ini program juga telah diperluas dengan penambahan dua folder baru, yaitu folder AST yang memuat struktur *Abstract Syntax Tree* beserta *visitor* dan konverternya (ASTNode.hpp, ASTNode.cpp, ASTConverter.hpp, ASTConverter.cpp, ASTVisitor.hpp), serta folder Semantic yang menangani tahapan analisis semantik (TypeRules.hpp, TypeRules.cpp, SymbolInfo.hpp, SymbolTable.cpp, SemanticAnalyzer.hpp, SemanticAnalyzer.cpp, ErrorReporter.hpp, ErrorReporter.cpp).
- Folder test berisi file pengujian, baik berupa data masukan (source code bahasa Arion) maupun data keluaran hasil pemrosesan token. Terdapat folder milestone-1, milestone-2, dan milestone-3 di dalamnya untuk menentukan *test case* per *milestone*.
- File Makefile berisi kumpulan instruksi kompilasi untuk mempermudah proses build source code C++ menjadi program executable secara otomatis.

2.2. Implementasi Program

1. ErrorReporter.hpp & ErrorReporter.cpp

ErrorReporter.hpp

```
class ErrorReporter {
private:
    std::vector<std::string> errors;
public:
    ErrorReporter() = default;
    void addErrors(int line, const std::string& message);
    bool hasErrors() const;
    const std::vector<std::string>& getErrors() const;
    void clear();
};
```

Pada ErrorReporter.hpp, implementasi berfungsi untuk mengelola dan menyimpan error (error handler) yang terjadi selama proses analisis kompilasi berlangsung. Bagian ini mendefinisikan kelas ErrorReporter yang membuat private atribut berupa struktur vektor string bernama errors sebagai tempat penyimpanan error. Selain itu, terdapat beberapa metode seperti addErrors untuk mencatat letak baris terjadinya error beserta detailnya, hasErrors untuk memudahkan verifikasi status keberadaan error dalam program, getErrors untuk mengakses dan mengembalikan vector errors, serta clear untuk mereset error sebelumnya.

ErrorReporter.cpp

```
void ErrorReporter::addErrors(int line, const std::string& message){
    INPUT : Integer nomor baris dan referensi string pesan kesalahan.
    OUTPUT : Penambahan pesan error ke dalam list.
    CARA KERJA :
    1. Menggabungkan nomor baris dan pesan kesalahan yang diberikan
        menjadi format string [SEMANTIC ERROR] Baris <line>: <message>
        lalu menambahkannya ke dalam struktur vektor errors.
}

bool ErrorReporter::hasErrors() const {
    INPUT : -
    OUTPUT : Boolean status keberadaan error.
    CARA KERJA :
    1. Mengembalikan nilai kebalikan dari errors.empty(). Akan bernilai
        true jika setidaknya satu pesan error tersimpan.
}
```

```

const std::vector<std::string>& ErrorReporter::getErrors() const{
    INPUT : -
    OUTPUT : Referensi read-only ke vektor berisi seluruh pesan error.
    CARA KERJA :
    1. Mengembalikan vektor errors apa adanya.
}

void ErrorReporter::clear(){
    INPUT : -
    OUTPUT : Daftar error dikosongkan.
    CARA KERJA :
    1. Memanggil .clear() pada vektor errors untuk menghapus seluruh isinya.
}

```

Pada ErrorReporter.cpp, implementasi berisi definisi fungsi-fungsi operasional untuk manajemen error, khususnya pada fase semantic analysis. Fungsi addErrors berfungsi memformat data letak baris dan detail pesan menjadi sebuah log. Untuk kebutuhan validasi, terdapat metode hasErrors untuk mengecek ketersediaan log secara cepat, serta getErrors untuk mengambil seluruh error yang telah terkumpul. Terakhir, terdapat fungsi clear yang mengimplementasikan pembersihan memori vektor agar instansiasi ErrorReporter dapat di-reset atau digunakan kembali dari awal yang bersih.

2. SemanticAnalyzer.hpp & SemanticAnalyzer.cpp

SemanticAnalyzer.hpp

```

class SemanticAnalyzer : public ASTVisitor {
private:
    SymbolTable symbolTable;
    ErrorReporter errorReporter;
    std::vector<int> activeBlocks;
    ExprType resolveTypeNode(ASTNode* typeNode);
    bool tryGetIntLiteral(ASTNode* node, int& value);
    ExprType resolveRecordFieldType(ASTNode* typeNode, const
std::string& fieldName);
    void report(ASTNode* node, const std::string& message);
public:
    SemanticAnalyzer() = default;
    void analyze(ASTNode* root);
    const ErrorReporter& getErrorReporter() const;
    const SymbolTable& getSymbolTable() const;
    void visitProgramNode(ProgramNode* node) override;
    void visitCompoundNode(CompoundNode* node) override;
    void visitConstDeclNode(ConstDeclNode* node) override;
}

```

```

void visitTypeDeclNode(TypeDeclNode* node) override;
void visitVarDeclNode(VarDeclNode* node) override;
void visitParamNode(ParamNode* node) override;
void visitSubprogramDeclNode(SubprogramDeclNode* node)
override;
void visitNamedTypeNode(NamedTypeNode* node) override;
void visitArrayTypeNode(ArrayTypeNode* node) override;
void visitRecordTypeNode(RecordTypeNode* node) override;
void visitRangeNode(RangeNode* node) override;
void visitEnumNode(EnumNode* node) override;
void visitAssignNode(AssignNode* node) override;
void visitIfNode(IfNode* node) override;
void visitCaseBlockNode(CaseBlockNode* node) override;
void visitCaseNode(CaseNode* node) override;
void visitWhileNode(WhileNode* node) override;
void visitRepeatNode(RepeatNode* node) override;
void visitForNode(ForNode* node) override;
void visitProcCallNode(ProcCallNode* node) override;
void visitVarNode(VarNode* node) override;
void visitArrayAccessNode(ArrayAccessNode* node) override;
void visitRecordAccessNode(RecordAccessNode* node) override;
void visitBinOpNode(BinOpNode* node) override;
void visitUnaryOpNode(UnaryOpNode* node) override;
void visitNumberNode(NumberNode* node) override;
void visitStringNode(StringNode* node) override;
void visitCharNode(CharNode* node) override;
};

```

Pada `SemanticAnalyzer.hpp`, kelas ini diimplementasikan dengan menerapkan rancangan Visitor Pattern (sebagai turunan dari kelas `ASTVisitor`). Kelas ini berfungsi sebagai untuk mengeksekusi tahap semantic analysis dengan mengintegrasikan instansiasi dari `SymbolTable` untuk scope beserta deklarasinya, serta `ErrorReporter` sebagai error handler. Untuk melakukan penelusuran struktur Abstract Syntax Tree (AST), kelas ini mendefinisikan metode `analyze` beserta serangkaian metode override `visit...` (seperti `visitProgramNode`, `visitBinOpNode`, dsb.).

SemanticAnalyzer.cpp

```

void SemanticAnalyzer::analyze(ASTNode* root) {
    INPUT : Pointer root ke root node dari Abstract Syntax Tree.
    OUTPUT : Memulai proses traversal semantic analysis.
    CARA KERJA :
    1. Melakukan validasi apakah root tidak null, kemudian memanggil
       metode accept(this) pada root tersebut untuk memulai penelusuran
       (traversal) struktur pohon menggunakan Visitor Pattern.
}

```

```

const ErrorReporter& SemanticAnalyzer::getErrorReporter() const {
    INPUT : -
    OUTPUT : Referensi read-only ke objek ErrorReporter.
    CARA KERJA :
    1. Mengembalikan variabel internal errorReporter yang menampung
        seluruh errors semantik selama proses analisis.
}

const SymbolTable& SemanticAnalyzer::getSymbolTable() const {
    INPUT : -
    OUTPUT : Referensi read-only ke objek SymbolTable.
    CARA KERJA :
    1. Mengembalikan variabel internal symbolTable yang menyimpan
        seluruh informasi deklarasi dan scope program.
}

void SemanticAnalyzer::report(ASTNode* node, const std::string&
message) {
    INPUT : Pointer ke ASTNode tempat error terjadi dan referensi
string message pesan kesalahan.
    OUTPUT : Error semantik.
    CARA KERJA :
    1. Mengekstraksi nomor baris (lineNum) dari node (jika tidak null),
        lalu meneruskannya bersama pesan error ke metode addErrors pada
        instansiasi errorReporter.
}

ExprType SemanticAnalyzer::resolveTypeNode(ASTNode* typeNode) {
    INPUT : Pointer ke ASTNode yang merepresentasikan sebuah tipe.
    OUTPUT : Nilai enumerasi ExprType
    CARA KERJA :
    1. Evaluasi menggunakan dynamic cast untuk mengetahui jenis node
        tipe (apakah NamedType, ArrayType, RecordType, Range, atau
        Enum).
    2. Jika berupa NamedType, metode akan mencari di Symbol Table,
        menandainya sebagai used, dan menelusuri secara rekursif jika
        tipe tersebut adalah alias dari tipe lain, lalu mengembalikan
        tipe primitif atau kompositnya.
}

bool SemanticAnalyzer::tryGetIntLiteral(ASTNode* node, int& value) {
    INPUT : Pointer ke ASTNode yang ingin dievaluasi dan referensi
integer value untuk menampung hasil.
    OUTPUT : Boolean status keberhasilan ekstraksi dan modifikasi
variabel value.
    CARA KERJA :
    1. Memeriksa apakah node adalah NumberNode bertipe integer.

```

```

    2. Jika ya, konversi string ke integer menggunakan std::strtol.
       Mengembalikan true dan menyimpan hasilnya jika konversi berhasil
       murni.
}

ExprType SemanticAnalyzer::resolveRecordFieldType(ASTNode* typeNode,
const std::string& fieldName) {
    INPUT : Pointer ke ASTNode definisi record dan string fieldName
    yang dicari.
    OUTPUT : ExprType dari field record yang dituju.
    CARA KERJA :
    1. Menelusuri definisi rekursif dari tipe.
    2. Jika node adalah RecordTypeNode, metode akan mengiterasi seluruh
       deklarasi variabel (field) di dalamnya.
    3. Jika nama field cocok dengan argumen, metode akan memanggil
       resolveTypeNode untuk mengembalikan tipe field tersebut.
}

ASTNode* SemanticAnalyzer::getTypeDefForExpression(ASTNode* expr) {
    INPUT : ASTNode* expr (ekspresi variabel/array/record).
    OUTPUT : ASTNode* typeDef atau nullptr.
    CARA KERJA :
    1. Jika expr null, langsung kembalikan nullptr.
    2. Jika expr adalah VarNode, lookup di symbolTable lalu kembalikan
       info→typeDef.
    3. Jika expr adalah ArrayAccessNode, cari typeDef parent
       secara rekursif, lalu kembalikan elementType jika parent adalah
       ArrayTypeNode.
    4. Jika expr adalah RecordAccessNode, cari typeDef parent
       secara rekursif, lalu cari fieldName di RecordTypeNode dan
       kembalikan typeDef field yang cocok.
}

void SemanticAnalyzer::visitProgramNode(ProgramNode* node) {
    INPUT : Pointer ke ProgramNode.
    OUTPUT : Pendaftaran blok utama ke Symbol Table.
    CARA KERJA :
    1. Mengatur level scope (lev).
    2. Mendaftarkan identitas program ke Symbol Table dan membuat blok
       (btab) baru dengan createBlock.
    3. Memasukkan indeks blok ke dalam activeBlocks, lalu mengunjungi
       seluruh node deklarasi dan mainBlock secara rekursif.
    4. Setelah selesai, blok ditutup dan di-pop.
}

void SemanticAnalyzer::visitCompoundNode(CompoundNode* node) {
    INPUT : Pointer ke CompoundNode.
    OUTPUT : Kunjungan ke setiap statement di dalam blok kode.
}

```


CARA KERJA :

1. Menyimpan level *scope* saat ini ke dalam node, lalu melakukan iterasi dan memanggil `accept(this)` ke seluruh *statement* anak (*children*) di dalamnya.

}

`void SemanticAnalyzer::visitConstDeclNode(ConstDeclNode* node) {`

INPUT : Pointer ke `ConstDeclNode`.

OUTPUT : Pendaftaran identifier konstanta ke dalam *Symbol Table*.

CARA KERJA :

1. Mengevaluasi nilai node (*value*), kemudian membuat objek `SymbolInfo` dengan *kind* `CONSTANT` beserta tipe data nilainya.
2. Mendaftarkannya ke *Symbol Table* dan mencatat *error* jika *identifier* sudah pernah dideklarasikan di *scope* tersebut.

}

`void SemanticAnalyzer::visitTypeDeclNode(TypeDeclNode* node) {`

INPUT : Pointer ke `TypeDeclNode`.

OUTPUT : Pendaftaran alias tipe bentukan ke dalam *Symbol Table*.

CARA KERJA :

1. Mengevaluasi definisi tipe, menentukan tipe ekspresinya melalui `resolveTypeNode`, membuat objek `SymbolInfo` berjenis `TYPE`, dan menyimpannya di *Symbol Table*. Memunculkan *error* jika tipe dideklarasikan ulang.

}

`void SemanticAnalyzer::visitVarDeclNode(VarDeclNode* node) {`

INPUT : Pointer ke `VarDeclNode`.

OUTPUT : Mendaftarkan identifier variabel beserta informasi array ke *Symbol Table* dan anotasi *node*.

CARA KERJA :

1. Menyelesaikan tipe variabel tersebut.
2. Jika tipenya adalah *Array*, metode memanggil `createArray` di *Symbol Table* untuk menyimpan batas statis (*bounds*) dan tipe indeks/elemennya (*atab*).
3. Mengiterasi daftar nama variabel, menyimpannya ke *Symbol Table* sebagai `VARIABLE`, lalu menganotasikan indeks tabel (*tabIndex*), level, dan *exprType* pada *node* AST.

}

`void SemanticAnalyzer::visitParamNode(ParamNode* node) {`

INPUT : Pointer ke `ParamNode`.

OUTPUT : Mendaftarkan parameter fungsi ke dalam *Symbol Table*.

CARA KERJA :

1. Menentukan tipe ekspresi dari definisi parameter.
2. Mengiterasi nama-nama parameter lalu mendeklarasikan `SymbolInfo` berjenis `PARAMETER`. Memunculkan *error* jika ada duplikasi nama.

```

}

void SemanticAnalyzer::visitSubprogramDeclNode(SubprogramDeclNode*
node) {
    INPUT : Pointer ke SubprogramDeclNode.
    OUTPUT : Mendaftarkan blok subprogram, parameter, deklarasi
internal, dan evaluasi subprogram.
    CARA KERJA :
    1. Menentukan apakah subprogram adalah FUNCTION (dengan return
type) atau PROCEDURE (VOID), lalu mendeklarasikannya di Symbol
Table.
    2. Membuat blok baru untuk mencatat hirarki (btab), mendaftarkan
list parameter ke entitas subprogram tersebut, masuk ke scope
baru, menelusuri isi parameter, deklarasi, dan eksekusi body,
lalu menutup dan keluar dari scope blok subprogram.
}

void SemanticAnalyzer::visitNamedTypeNode(NamedTypeNode* node) {
    INPUT : Pointer ke NamedTypeNode.
    OUTPUT : Anotasi exprType pada node.
    CARA KERJA :
    1. Memanggil resolveTypeNode pada node ini sendiri untuk
menyelesaikan apakah ia tipe dasar atau referensi alias, lalu
menyimpan hasilnya pada atribut exprType.
}

void SemanticAnalyzer::visitArrayTypeNode(ArrayTypeNode* node) {
    INPUT : Pointer ke ArrayTypeNode.
    OUTPUT : Evaluasi indeks/elemen dan anotasi tipe ARRAY.
    CARA KERJA :
    1. Mengunjungi node indexType dan elementType untuk mengevaluasi
semantiknya, kemudian menetapkan nilai exprType node menjadi
ARRAY.
}

void SemanticAnalyzer::visitRecordTypeNode(RecordTypeNode* node) {
    INPUT : Pointer ke RecordTypeNode.
    OUTPUT : Evaluasi fields dan anotasi tipe RECORD.
    CARA KERJA :
    1. Melakukan iterasi kunjungan untuk mengevaluasi seluruh ASTNode
fields di dalam record, lalu menetapkan nilai exprType node ini
menjadi RECORD.
}

void SemanticAnalyzer::visitRangeNode(RangeNode* node) {
    INPUT : Pointer ke RangeNode.
    OUTPUT : Evaluasi batas range dan anotasi tipe SUBRANGE.
}

```

```

    CARA KERJA :
    1. Mengunjungi node batas bawah (lowerBound) dan batas atas
       (upperBound), lalu menetapkan tipe node ini sebagai SUBRANGE.
}

void SemanticAnalyzer::visitEnumNode(EnumNode* node) {
    INPUT : Pointer ke EnumNode.
    OUTPUT : Anotasi tipe ENUM.
    CARA KERJA :
    1. Menetapkan nilai atribut exprType menjadi ENUM.
}

void SemanticAnalyzer::visitAssignNode(AssignNode* node) {
    INPUT : Pointer ke AssignNode.
    OUTPUT : Validasi type compatibility untuk instruksi assignment.
    CARA KERJA :
    1. Mengevaluasi node target (variabel penerima) dan value (ekspresi
       nilai).
    2. Membandingkan tipe keduanya menggunakan aturan
       TypeRules::isAssignable.
    3. Jika terjadi pelanggaran kompatibilitas (misal: assign Char ke
       Integer), maka error akan dicatat. Tipe node diatur menjadi
       VOID.
}

void SemanticAnalyzer::visitIfNode(IfNode* node) {
    INPUT : Pointer ke IfNode.
    OUTPUT : Penelusuran blok kondisi logika bersyarat.
    CARA KERJA :
    1. Menelusuri cabang kondisi evaluasi, blok thenBlock jika true,
       serta cabang elseBlock jika disediakan.
}

void SemanticAnalyzer::visitCaseBlockNode(CaseBlockNode* node) {
    INPUT : Pointer ke CaseBlockNode.
    OUTPUT : Penelusuran opsi case dan statement pembawanya.
    CARA KERJA :
    1. Mengiterasi kunjungan pada seluruh daftar nilai konstanta opsi
       case, lalu mengunjungi node statement yang akan dieksekusi bila
       case tersebut terpenuhi.
}

void SemanticAnalyzer::visitCaseNode(CaseNode* node) {
    INPUT : Pointer ke CaseNode.
    OUTPUT : Penelusuran kondisi induk case dan anak cabangnya.
    CARA KERJA :

```

```

    1. Mengunjungi node kondisi (nilai yang dicocokkan), lalu
       mengiterasi kunjungan ke seluruh CaseBlockNode di dalamnya.
}

void SemanticAnalyzer::visitWhileNode(WhileNode* node) {
    INPUT : Pointer ke WhileNode.
    OUTPUT : Penelusuran instruksi perulangan while.
    CARA KERJA :
    1. Mengunjungi node kondisional untuk dievaluasi semantiknya,
       kemudian menelusuri loopBlock.
}

void SemanticAnalyzer::visitRepeatNode(RepeatNode* node) {
    INPUT : Pointer ke RepeatNode.
    OUTPUT : Penelusuran instruksi perulangan repeat-until.
    CARA KERJA :
    1. Mengiterasi dan mengevaluasi seluruh statements di dalam blok
       perulangan secara berurutan, kemudian menelusuri node
       kondisional penghentian (until).
}

void SemanticAnalyzer::visitForNode(ForNode* node) {
    INPUT : Pointer ke ForNode.
    OUTPUT : Validasi variabel iterator dan penelusuran perulangan for.
    CARA KERJA :
    1. Memeriksa keberadaan iteratorName di Symbol Table.
    2. Mencatat error jika belum dideklarasikan atau jika tipenya bukan
       Integer/Subrange.
    3. Mengunjungi nilai awalan, akhiran, dan blok loop.
    4. Memastikan tipe dari nilai start dan end cocok untuk di-assign
       ke variabel iterator menggunakan pengecekan
       TypeRules::isAssignable.
}

void SemanticAnalyzer::visitProcCallNode(ProcCallNode* node) {
    INPUT : Pointer ke ProcCallNode.
    OUTPUT : Validasi argumen dan anotasi pemanggilan prosedur/fungsi.
    CARA KERJA :
    1. Mengunjungi seluruh node argumen.
    2. Mencari nama subprogram di tabel simbol dan mencatat letak level
       dan indeksinya di tabel.
    3. Jika bukan subprogram built-in I/O (seperti write/read), sistem
       akan memvalidasi kuantitas argumen terhadap definisi parameter
       aktual.
    4. Membandingkan satu per satu kompatibilitas tipe antara argumen
       yang dikirim dengan deklarasi parameter.
}

```

```

void SemanticAnalyzer::visitVarNode(VarNode* node) {
    INPUT : Pointer ke VarNode.
    OUTPUT : Pengecekan status deklarasi identifier dan anotasi tipe variabel.
    CARA KERJA :
    1. Mencari identifier variabel menggunakan symbolTable.lookup().
    2. Jika tidak ditemukan, error undeclared dilontarkan.
    3. Jika ada, maka variabel akan ditandai isUsed, dan properti tabIndex, lev, serta exprType di node AST akan diisi berdasarkan data dari Symbol Table.
}

void SemanticAnalyzer::visitArrayAccessNode(ArrayAccessNode* node) {
    INPUT : Pointer ke ArrayAccessNode.
    OUTPUT : Validasi tipe indeks dan anotasi elemen array yang diakses.
    CARA KERJA :
    1. Mengunjungi variabel array dan seluruh argumen indeksinya.
    2. Mengecek apakah tipe indeks bernilai valid (hanya Integer/Subrange/Enum/Char).
    3. Mengambil typeDef array lewat getTypeDefForExpression, termasuk resolusi NamedTypeNode.
    4. Menetapkan exprType dari elementType dan mengecek out-of-bounds jika range statis dan indeks literal.
}

void SemanticAnalyzer::visitRecordAccessNode(RecordAccessNode* node) {
    INPUT : Pointer ke RecordAccessNode.
    OUTPUT : Resolusi field access dan anotasi tipe nilai record.
    CARA KERJA :
    1. Mengunjungi variabel pembawa (recordVar).
    2. Mengambil typeDef record lewat getTypeDefForExpression (bukan hanya lookup VarNode).
    3. Menelusuri field di RecordTypeNode dan mencocokkan fieldName.
    4. Menetapkan exprType sesuai tipe field atau mengeluarkan error jika tidak ditemukan.
}

void SemanticAnalyzer::visitBinOpNode(BinOpNode* node) {
    INPUT : Pointer ke BinOpNode.
    OUTPUT : Evaluasi ekspresi biner dan anotasi tipe hasil operasi.
    CARA KERJA :
    1. Menelusuri semantik sisi operator kiri (left) dan kanan (right).
    2. Meneruskan tipe keduanya ke rutin validasi TypeRules::resultOfBinary untuk menentukan kompatibilitas dan mengembalikan tipe keluaran agregatnya.
}

```

```

    3. Jika incompatible, munculkan error.
}

void SemanticAnalyzer::visitUnaryOpNode(UnaryOpNode* node) {
    INPUT : Pointer ke UnaryOpNode.
    OUTPUT : Evaluasi ekspresi unary dan anotasi tipe hasil.
    CARA KERJA :
    1. Mengunjungi node operand, kemudian meneruskan tipenya kepada
       rutinitas TypeRules::resultOfUnary.
    2. Menetapkan exprType menjadi return tipe hasil atau mencatat
       error jika operator tersebut tidak sah diaplikasikan pada tipe
       datanya.
}

void SemanticAnalyzer::visitNumberNode(NumberNode* node) {
    INPUT : Pointer ke NumberNode.
    OUTPUT : Anotasi exprType konstanta literal angka.
    CARA KERJA :
    1. Mengevaluasi bendera boolean isReal.
    2. Menetapkan type node menjadi tipe REAL apabila bernilai true,
       atau INTEGER jika false.
}

void SemanticAnalyzer::visitStringNode(StringNode* node) {
    INPUT : Pointer ke StringNode.
    OUTPUT : Anotasi exprType literal teks/string.
    CARA KERJA :
    1. Mengganti status primitif exprType di node AST secara langsung
       menjadi tipe STRING.
}

void SemanticAnalyzer::visitCharNode(CharNode* node) {
    INPUT : Pointer ke CharNode.
    OUTPUT : Anotasi exprType literal karakter.
    CARA KERJA :
    1. Mengganti status primitif exprType di node AST secara langsung
       menjadi tipe CHAR.
}

```

Pada `SemanticAnalyzer.cpp`, file ini berisi implementasi dari penelusuran struktur AST yang menerapkan arsitektur Visitor Pattern. Setiap jenis simpul (node) dievaluasi, meliputi operasi registrasi dan pencarian identifier (lookup), validasi penetapan batas array, serta pengelolaan hierarki ruang lingkup (scope).

3. SymbolInfo.hpp

SymbolInfo.hpp

```
enum class SymbolKind {
    VARIABLE,
    CONSTANT,
    FUNCTION,
    PROCEDURE,
    TYPE,
    PARAMETER,
    PROGRAM
};

struct ParamInfo {
    std::string name;
    ExprType type = ExprType::UNKNOWN;
    bool isVarParam = false;
    int declLine = 0;
};

struct SymbolInfo {
    std::string name;
    SymbolKind kind;
    ExprType type = ExprType::UNKNOWN;
    int tabIndex = -1;
    int level = -1;
    int declLine = 0;
    std::vector<ParamInfo> parameters;
    ASTNode* typeDef = nullptr;
    bool isParameter = false;
    bool isUsed = false;
    int blockIndex = -1;
    int arrayIndex = -1;
    SymbolInfo() = default;
    SymbolInfo(const std::string& name, SymbolKind kind) : name(name),
kind(kind) {}
};
```

File SymbolInfo.hpp berfungsi mendefinisikan blue-print dari data yang dapat masuk ke dalam Symbol Table. Di sini terdapat enum SymbolKind untuk membedakan sifat entitas dalam memori, mendefinisikan struk ParamInfo untuk mengetahui parameter prosedur/fungsi, dan struk SymbolInfo yang menjadi wadah utama perekaman sebuah simbol, merangkum jenis deklarasi, ruang alamat lingkup (level/index), jenis data (ExprType), pointer ke tabel definisi (typeDef).

4. SymbolTable.hpp & SymbolTable.cpp

SymbolTable.hpp

```
struct BlockInfo {
    int blockIndex = -1;
    std::string name;
    int parentBlock = -1;
    int level = -1;
    int startTabIndex = -1;
    int endTabIndex = -1;
};

struct ArrayInfo {
    int arrayIndex = -1;
    ExprType elementType = ExprType::UNKNOWN;
    ExprType indexType = ExprType::UNKNOWN;
    ASTNode* indexTypeNode = nullptr;
    ASTNode* elementTypeNode = nullptr;
    int lowerBound = 0;
    int upperBound = 0;
    bool hasStaticBounds = false;
};

class SymbolTable {
private:
    std::vector<std::unordered_map<std::string, SymbolInfo>> scopes;
    std::vector<SymbolInfo> symbolsList;
    int nextIndex;
    std::vector<BlockInfo> blocksList;
    std::vector<ArrayInfo> arraysList;
    int nextBlockIndex = 0;
    int nextArrayIndex = 0;
public:
    SymbolTable();
    void enterScope();
    void exitScope();
    bool declare(const std::string& name, SymbolInfo info);
    SymbolInfo* lookup(const std::string& name);
    SymbolInfo* lookUpCurrent(const std::string& name);
    int currentLevel() const;
    int nextTabIndex() const;
    const std::vector<SymbolInfo>& allSymbols() const;
    int createBlock(const std::string& name, int parentBlock, int
level, int startTabIndex);
    void closeBlock(int blockIndex, int endTabIndex);
    const BlockInfo* getBlock(int blockIndex) const;
    const std::vector<BlockInfo>& getAllBlocks() const;
    int createArray(
        ExprType indexType,
        ExprType elementType,
        ASTNode* indexTypeNode = nullptr,
```



```

        ASTNode* elementTypeNode = nullptr,
        bool hasStaticBounds = false,
        int lowerBound = 0,
        int upperBound = 0
    );
    const ArrayInfo* getArray(int arrayIndex) const;
    const std::vector<ArrayInfo>& getAllArrays() const;
};

```

SymbolTable.cpp

```

SymbolTable::SymbolTable()
    INPUT : -
    OUTPUT : Objek memori dari tabel simbol.
    CARA KERJA :
    1. Mengawali indeks di 0.
    2. Otomatis masuk ke scope awal, dan mendeklarasikan
       tipe-tipe/fungsi bawaan compiler secara otomatis (seperti
       integer, writeln, length, konstanta true/false) sebagai built-in
       mapping.
}

void SymbolTable::enterScope() / exitScope()
    INPUT : -
    OUTPUT : Menambahkan layer map ke ujung (masuk) dan menghapusnya.
    CARA KERJA :
    1. Menggunakan fungsi emplace_back() pada vektor scopes untuk
       menambahkan lapisan unordered_map saat inisiasi, atau di-pop
       saat ditutup/keluar dari scope.
}

bool SymbolTable::declare(const std::string& name, SymbolInfo info)
    INPUT : Nama dan kumpulan metadata informasinya.
    OUTPUT : boolean.
    CARA KERJA :
    1. Meninjau nama tersebut di scope teratas (terakhir).
    2. Jika sudah ada, pengembalian false dilakukan dan error tercatat,
    3. Jika belum, metadata ini dialokasikan memori tabIndex dan
       levelnya lalu disimpan pada map dan daftar general.
}

```

```

SymbolInfo* SymbolTable::lookup(const std::string& name)
    INPUT : String nama identifier.
    OUTPUT : Pointer referensi ke simbol info yang dicari, nullptr jika
    lenyap/tidak terdaftar.
    CARA KERJA :
    1. Dilakukan iterasi terbalik (dari scope paling dalam / akhir
    vektor menuju ke yang paling atas/root).
    2. Jika tertangkap duluan akan mengembalikan nilainya.
    3. lookUpCurrent berfungsi identik namun terbatas pada scope paling
    ujung tok.
}

int SymbolTable::createBlock(...) / int SymbolTable::createArray(...)
    INPUT : Atribut-atribut mendalam parameter pendukung block/program
    atau pembatasan parameter (lower, upper batas index array).
    OUTPUT : Integer token dari indeks referensi lokasi penyimpanannya.
    CARA KERJA :
    1. Membuat Struct BlockInfo maupun ArrayInfo sesuai informasi yang
    didapat dan push menuju vektor, kemudian di-Return ID-nya.
}

```

Pada SymbolTable.cpp, file ini berisi implementasi struktur data yang bertanggung jawab untuk mencatat dan mengelola setiap deklarasi simbol program, meliputi variabel, subprogram (prosedur dan fungsi), serta tipe data. Selain itu, kelas ini juga memiliki method untuk memelihara tabel metadata terpisah, yakni tabel Array (atab) dan tabel Blok (btab).

5. TypeRules.hpp & TypeRules.cpp

TypeRules.hpp

```

namespace TypeRules {
    bool isNumeric(ExprType t);
    bool isInteger(ExprType t);
    bool isReal(ExprType t);
    bool isBoolean(ExprType t);
    bool isChar(ExprType t);
    bool isString(ExprType t);
    bool canImplicitCast(ExprType from, ExprType to);
    bool isAssignable(ExprType lhs, ExprType rhs);
    ExprType resultOfBinary(const std::string& op, ExprType left,

```

```

ExprType right);
    ExprType resultOfUnary(const std::string& op, ExprType operand);
    ExprType resultOfRelational(ExprType left, ExprType right);
    ExprType resultOfLogical(ExprType left, ExprType right);
    bool isValidArrayType(ExprType idxType);
}

```

Pada `TypeRules.hpp`, implementasi didesain menggunakan namespace tersendiri yang bersifat stateless. Namespace ini secara khusus mendefinisikan aturan-aturan semantik untuk memvalidasi operasi-operasi kompilator, seperti penentuan kelayakan instruksi assignment, serta penentuan tipe data akhir (result type) yang dihasilkan ketika suatu operan dievaluasi bersama operator aritmatika maupun relasional.

TypeRules.cpp

```

bool TypeRules::isNumeric(ExprType t) {
    INPUT : Enumerasi ExprType t yang merepresentasikan tipe data.
    OUTPUT : Boolean status.
    CARA KERJA :
    1. Mengevaluasi secara logis apakah tipe t merupakan `INTEGER` atau `REAL`.
    2. Mengembalikan nilai true jika cocok dengan salah satunya.
}

bool TypeRules::canImplicitCast(ExprType from, ExprType to) {
    INPUT : Dua Enumerasi ExprType dari ruas kiri (target) dan ruas kanan (asal).
    OUTPUT : Boolean status.
    CARA KERJA :
    1. Mengembalikan nilai true jika kedua tipe sudah identik sedari awal.
    2. Mendefinisikan type promotion, di mana tipe asal berupa `INTEGER` diizinkan untuk dikonversi secara implisit menjadi `REAL`.
    3. Jika tidak memenuhi kondisi tersebut, fungsi akan mengembalikan false.
}

bool TypeRules::isAssignable(ExprType lhs, ExprType rhs) {
    INPUT : Dua Enumerasi ExprType dari ruas kiri (target) dan ruas kanan (asal).

```

OUTPUT : Boolean kelayakan operasi *assignment* (penugasan).

CARA KERJA :

1. Melakukan deteksi awal: jika salah satu tipe bernilai ``UNKNOWN``, langsung mengembalikan *false*.
2. Jika tipenya sama persis, mengembalikan *true*.
3. Jika berbeda, fungsionalitas validasi didelegasikan kepada metode ``canImplicitCast`` untuk memeriksa apakah nilai ``rhs`` sah ditampung di ``lhs`` melalui konversi.

}

```
ExprType TypeRules::resultOfBinary(const std::string& op, ExprType left, ExprType right) {
```

INPUT : String Simbol operator (+, and, <, dsb), bersama Tipe

ExprType operand kiri & kanannya.

OUTPUT : ExprType hasil deduksi akhir atau ExprType::UNKNOWN

CARA KERJA :

1. Menggunakan seleksi IF.
2. Untuk operator aritmatika komputasi, fungsi memvalidasi apakah kedua operan bertipe numerik.
3. Untuk operator logika kondisional (``and``, ``or``), mensyaratkan kedua operan bernilai ``BOOLEAN`` dan menghasilkan ``BOOLEAN``.
4. Untuk operator pembandingan (relasional), pengecekan diserahkan kepada ``resultOfRelational()`` dan mengembalikan ``BOOLEAN`` jika lolos uji kompatibilitas.

}

```
ExprType TypeRules::resultOfUnary(const std::string& op, ExprType operand) {
```

INPUT : String Simbol operator (+, and, <, dsb), bersama Tipe

ExprType operand kiri & kanannya.

OUTPUT : ExprType hasil deduksi akhir atau ExprType::UNKNOWN

CARA KERJA :

1. Jika operator yang digunakan adalah (``+`` atau ``-``), fungsi memastikan operan adalah nilai numerik lalu mengembalikan tipe asli operan tersebut.
2. Jika operatornya adalah negasi (``not``), fungsi memverifikasi operan harus berjenis ``BOOLEAN`` dan hasil akhirnya adalah ``BOOLEAN``.

}

```
ExprType TypeRules::resultOfRelational(ExprType left, ExprType right) {
```

```

    INPUT : String Simbol operator (+, and, <, dsb), bersama Tipe
    ExprType operand kiri & kanannya.
    OUTPUT : ExprType hasil deduksi akhir atau ExprType::UNKNOWN
    CARA KERJA :
    1. Memeriksa validitas komparasi antar tipe data.
    2. Perbandingan dianggap sah dan akan mereturn `BOOLEAN` jika kedua
       tipe adalah numerik
    3. Perbandingan juga mereturn `BOOLEAN` jika kedua tipe identik dan
       merupakan salah satu dari tipe `CHAR`, `STRING`, atau bentukan
       `ENUM`.
    4. Perbandingan di luar aturan tersebut dianggap ilegal dan
       mengembalikan `UNKNOWN`.
}

ExprType TypeRules::resultOfLogical(ExprType left, ExprType right) {
    INPUT : String Simbol operator (+, and, <, dsb), bersama Tipe
    ExprType operand kiri & kanannya.
    OUTPUT : ExprType hasil deduksi akhir atau ExprType::UNKNOWN
    CARA KERJA :
    1. Operasi logical murni hanya sah jika tipe operan kiri dan kanan
       sama-sama terverifikasi sebagai `BOOLEAN`.
}

bool TypeRules::isValidArrayType(ExprType idxType) {
    INPUT : `ExprType idxType`
    OUTPUT : Boolean penentu kelayakan tipe indeks.
    CARA KERJA :
    1. Memastikan bahwa tipe ekspresi pemanggil indeks adalah tipe data
       yang bersifat diskrit, ordinal, dan dapat dihitung.
    2. Fungsi mengembalikan true jika indeks bertipe `INTEGER`, `CHAR`,
       bentukan `ENUM`, atau spesifikasi `SUBRANGE`.
}

```

Pada `TypeRules.cpp`, implementasi dirancang sebagai kumpulan fungsi utilitas murni (stateless) yang bertugas secara spesifik sebagai pusat validasi kompatibilitas semantik antar tipe data. File ini memverifikasi operasi-operasi di dalam Abstract Syntax Tree (seperti implicit casting, penugasan/ assignment, dan evaluasi ekspresi operator) dengan mengembalikan hasil deduksi tipe akhir jika ekspresi bernilai benar. Sebaliknya, akan mengembalikan status UNKNOWN apabila terjadi pelanggaran kompatibilitas atau ketidakcocokan tipe (type mismatch) antar operan pada kode sumber.

6. ASTConverter.hpp & ASTConverter.cpp

ASTConverter.hpp

```
class ASTConverter{
public :
    ASTNode* build(ParseNode* parseRoot);
private:
    ASTNode* convertProgram(ParseNode* node);
    ASTNode* convertCompoundStatement(ParseNode* node);
    vector<ASTNode*> convertStatementList(ParseNode* node);

    vector<ASTNode*> convertDeclarationPart(ParseNode* node);
    vector<ASTNode*> convertConstDeclaration(ParseNode* node);
    vector<ASTNode*> convertTypeDeclaration(ParseNode* node);
    vector<ASTNode*> convertVarDeclaration(ParseNode* node);
    vector<string> convertIdentifierList(ParseNode* node);
    ASTNode* convertSubprogramDeclaration(ParseNode* node);
    vector<ASTNode*> convertFormalParameterList(ParseNode* node);

    ASTNode* convertType(ParseNode* node);
    ASTNode* convertConstant(ParseNode* node);
    ASTNode* convertArrayType(ParseNode* node);
    ASTNode* convertRange(ParseNode* node);
    ASTNode* convertEnumerated(ParseNode* node);
    ASTNode* convertRecordType(ParseNode* node);

    ASTNode* convertStatement(ParseNode* node);
    ASTNode* convertAssignment(ParseNode* node);
    ASTNode* convertIf(ParseNode* node);
    ASTNode* convertCase(ParseNode* node);
    ASTNode* convertCaseBlock(ParseNode* node);
    ASTNode* convertWhile(ParseNode* node);
    ASTNode* convertRepeat(ParseNode* node);
    ASTNode* convertFor(ParseNode* node);
    ASTNode* convertProcCall(ParseNode* node);

    ASTNode* convertVariable(ParseNode* node);
    vector<ASTNode*> convertIndexList(ParseNode* node);
    vector<ASTNode*> convertParameterList(ParseNode* node);

    ASTNode* convertExpression(ParseNode* node);
```

```

    ASTNode* convertSimpleExpression(ParseNode* node);
    ASTNode* convertTerm(ParseNode* node);
    ASTNode* convertFactor(ParseNode* node);
}

```

Pada `ASTConverter.hpp`, file ini mendefinisikan kelas `ASTConverter` sebagai antarmuka utama untuk mengubah Parse Tree (`ParseNode`) menjadi Abstract Syntax Tree (`ASTNode`). Di dalamnya tercantum deklarasi metode konversi untuk setiap bagian grammar penting seperti struktur program, deklarasi, tipe data, statement, serta ekspresi. Dengan pembagian fungsi yang jelas per non-terminal, header ini menjadi peta proses transformasi dari struktur sintaks mentah ke AST yang lebih ringkas dan siap dianalisis secara semantik.

ASTConverter.cpp

```

ASTNode* ASTConverter::build(ParseNode* parseRoot) {
    INPUT  : Root ParseNode hasil parser.
    OUTPUT : ASTNode* root AST atau nullptr.
    CARA KERJA :
    1. Mengecek apakah parseRoot null.
    2. Memastikan root bernama <program>.
    3. Memanggil convertProgram jika valid, selain itu mengembalikan nullptr.
}

ASTNode* ASTConverter::convertProgram(ParseNode* node) {
    INPUT  : ParseNode <program>.
    OUTPUT : ProgramNode berisi nama program, deklarasi, dan blok utama.
    CARA KERJA :
    1. Mengambil nama program dari <program-header>.
    2. Mengonversi <declaration-part> menjadi daftar deklarasi AST.
    3. Mengonversi <compound-statement> menjadi main block.
    4. Membentuk dan mengembalikan ProgramNode.
}

ASTNode* ASTConverter::convertCompoundStatement(ParseNode* node) {
    INPUT  : ParseNode <compound-statement>.
    OUTPUT : CompoundNode berisi list statement.
    CARA KERJA :
    1. Mengonversi <statement-list> menjadi vector ASTNode.
    2. Membungkus hasil ke CompoundNode.
}

vector<ASTNode*> ASTConverter::convertStatementList(ParseNode* node) {

```

```

    INPUT  : ParseNode <statement-list>.
    OUTPUT : Vector ASTNode untuk setiap statement.
    CARA KERJA :
    1. Mengiterasi semua child <statement-list>.
    2. Memilih child bertipe <statement>.
    3. Mengonversi tiap statement dengan method convertStatement() dan
        menambahkan yang tidak null.
}

vector<ASTNode*> ASTConverter::convertDeclarationPart(ParseNode* node)
{
    INPUT  : ParseNode <declaration-part>.
    OUTPUT : Vector ASTNode deklarasi (const / type / var /
        subprogram).
    CARA KERJA :
    1. Mengiterasi anak <declaration-part>.
    2. Merutekan sesuai jenis deklarasi (const / type / var /
        subprogram).
    3. Menggabungkan semua hasil ke satu vector.
}

vector<ASTNode*> ASTConverter::convertConstDeclaration(ParseNode*
node) {
    INPUT  : ParseNode <const-declaration>.
    OUTPUT : Vector ConstDeclNode.
    CARA KERJA :
    1. Iterasi per 4 token (ident, eql, constant, semicolon).
    2. Mengonversi constant menjadi node literal.
    3. Membuat ConstDeclNode untuk tiap pasangan.
}

vector<ASTNode*> ASTConverter::convertTypeDeclaration(ParseNode* node)
{
    INPUT  : ParseNode <type-declaration>.
    OUTPUT : Vector TypeDeclNode.
    CARA KERJA :
    1. Iterasi per 4 token (ident, eql, type, semicolon).
    2. Mengonversi type menjadi ASTNode tipe.
    3. Membuat TypeDeclNode untuk tiap definisi.
}

vector<ASTNode*> ASTConverter::convertVarDeclaration(ParseNode* node)
{
    INPUT  : ParseNode <var-declaration>.
    OUTPUT : Vector VarDeclNode.
    CARA KERJA :
    1. Mengambil identifier-list sebagai daftar nama.
    2. Mengonversi type untuk setiap grup variabel.
}

```



```

    3. Membuat VarDeclNode dan mengumpulkan hasilnya.
}

vector<string> ASTConverter::convertIdentifierList(ParseNode* node) {
    INPUT : ParseNode <identifier-list>.
    OUTPUT : Vector string nama identifier.
    CARA KERJA :
    1. Mengambil ident pada indeks genap (melewati comma).
    2. Mengumpulkan semua nama ke vector.
}

ASTNode* ASTConverter::convertSubprogramDeclaration(ParseNode* node) {
    INPUT : ParseNode <subprogram-declaration>.
    OUTPUT : SubprogramDeclNode (fungsi/prosedur).
    CARA KERJA :
    1. Menentukan apakah deklarasi fungsi atau prosedur.
    2. Mengambil nama subprogram dan parameter jika ada.
    3. Mengambil return type jika function.
    4. Mengonversi block menjadi deklarasi lokal dan body.
    5. Membentuk SubprogramDeclNode.
}

vector<ASTNode*> ASTConverter::convertFormalParameterList(ParseNode*
node) {
    INPUT : ParseNode <formal-parameter-list>.
    OUTPUT : Vector ParamNode.
    CARA KERJA :
    1. Mengiterasi setiap parameter-group.
    2. Mengonversi identifier-list dan tipe param.
    3. Membuat ParamNode untuk tiap grup.
}

ASTNode* ASTConverter::convertType(ParseNode* node) {
    INPUT : ParseNode <type>.
    OUTPUT : Node tipe sesuai grammar.
    CARA KERJA :
    1. Membaca anak pertama untuk menentukan jenis tipe.
    2. Memanggil konversi spesifik (array/range/enum/record) bila
        cocok.
    3. Jika hanya ident, membuat NamedTypeNode.
}

ASTNode* ASTConverter::convertArrayType(ParseNode* node) {
    INPUT : ParseNode <array-type>.
    OUTPUT : ArrayTypeNode.
    CARA KERJA :
    1. Mengonversi indexType (range atau named type).
    2. Mengonversi elementType.
}

```

```

    3. Membentuk ArrayTypeNode.
}

ASTNode* ASTConverter::convertRange(ParseNode* node) {
    INPUT  : ParseNode <range>.
    OUTPUT : RangeNode.
    CARA KERJA :
    1. Mengonversi constant batas bawah.
    2. Mengonversi constant batas atas.
    3. Membentuk RangeNode.
}

ASTNode* ASTConverter::convertEnumerated(ParseNode* node) {
    INPUT  : ParseNode <enumerated>.
    OUTPUT : EnumNode.
    CARA KERJA :
    1. Mengambil semua ident di dalam tanda kurung.
    2. Mengumpulkan ke vector string.
    3. Membentuk EnumNode.
}

ASTNode* ASTConverter::convertRecordType(ParseNode* node) {
    INPUT  : ParseNode <record-type>.
    OUTPUT : RecordTypeNode.
    CARA KERJA :
    1. Mengiterasi field-list.
    2. Mengonversi field-part menjadi VarDeclNode.
    3. Mengumpulkan semua field ke RecordTypeNode.
}

ASTNode* ASTConverter::convertConstant(ParseNode* node) {
    INPUT  : ParseNode <constant>.
    OUTPUT : Node literal atau VarNode.
    CARA KERJA :
    1. Mengecek unary plus/minus dan menyimpan tanda.
    2. Mengambil token nilai dan jenis literal.
    3. Membentuk CharNode/StringNode/NumberNode sesuai tipe token.
    4. Jika ident, membentuk VarNode.
}

ASTNode* ASTConverter::convertStatement(ParseNode* node) {
    INPUT  : ParseNode <statement>.
    OUTPUT : Node statement spesifik atau nullptr.
    CARA KERJA :
    1. Mengecek apakah statement kosong.
    2. Mengidentifikasi jenis statement dari child pertama.
    3. Memanggil fungsi konversi yang sesuai dengan jenis statement.
}

```

```

ASTNode* ASTConverter::convertAssignment(ParseNode* node) {
    INPUT  : ParseNode <assignment-statement>.
    OUTPUT : AssignNode.
    CARA KERJA :
    1. Mengonversi <variable> sebagai target.
    2. Mengonversi <expression> sebagai value.
    3. Membentuk AssignNode.
}

```

```

ASTNode* ASTConverter::convertIf(ParseNode* node) {
    INPUT  : ParseNode <if-statement>.
    OUTPUT : IfNode.
    CARA KERJA :
    1. Mengonversi ekspresi kondisi.
    2. Mengonversi then-block.
    3. Mengonversi else-block jika ada.
    4. Membentuk IfNode.
}

```

```

ASTNode* ASTConverter::convertCase(ParseNode* node) {
    INPUT  : ParseNode <case-statement>.
    OUTPUT : CaseNode.
    CARA KERJA :
    1. Mengonversi ekspresi kondisi.
    2. Mengonversi case-block pertama.
    3. Menelusuri case-block bersarang hingga habis.
    4. Mengumpulkan semua CaseBlockNode ke CaseNode.
}

```

```

ASTNode* ASTConverter::convertCaseBlock(ParseNode* node) {
    INPUT  : ParseNode <case-block>.
    OUTPUT : CaseBlockNode.
    CARA KERJA :
    1. Mengambil semua <constant> sebagai label kasus.
    2. Mengonversi satu <statement> sebagai aksi.
    3. Membentuk CaseBlockNode.
}

```

```

ASTNode* ASTConverter::convertWhile(ParseNode* node) {
    INPUT  : ParseNode <while-statement>.
    OUTPUT : WhileNode.
    CARA KERJA :
    1. Mengonversi ekspresi kondisi.
    2. Mengonversi compound-statement sebagai loop body.
    3. Membentuk WhileNode.
}

```

```

ASTNode* ASTConverter::convertRepeat(ParseNode* node) {
    INPUT  : ParseNode <repeat-statement>.
    OUTPUT : RepeatNode.
    CARA KERJA :
    1. Mengonversi statement-list menjadi body.
    2. Mengonversi ekspresi until sebagai kondisi.
    3. Membentuk RepeatNode.
}

ASTNode* ASTConverter::convertFor(ParseNode* node) {
    INPUT  : ParseNode <for-statement>.
    OUTPUT : ForNode.
    CARA KERJA :
    1. Mengambil nama iterator.
    2. Mengonversi ekspresi start dan end.
    3. Menentukan arah to/downto.
    4. Mengonversi compound-statement sebagai body.
    5. Membentuk ForNode.
}

ASTNode* ASTConverter::convertProcCall(ParseNode* node) {
    INPUT  : ParseNode <procedure/function-call>.
    OUTPUT : ProcCallNode.
    CARA KERJA :
    1. Mengambil nama prosedur/fungsi.
    2. Mengonversi parameter-list jika ada.
    3. Membentuk ProcCallNode.
}

vector<ASTNode*> ASTConverter::convertParameterList(ParseNode* node) {
    INPUT  : ParseNode <parameter-list>.
    OUTPUT : Vector ASTNode argumen.
    CARA KERJA :
    1. Mengonversi setiap <expression> yang dipisah comma.
    2. Mengumpulkan argumen ke vector.
}

ASTNode* ASTConverter::convertVariable(ParseNode* node) {
    INPUT  : ParseNode <variable>.
    OUTPUT : VarNode atau node akses bertingkat.
    CARA KERJA :
    1. Membuat VarNode dari ident dasar.
    2. Mengiterasi component-variable.
    3. Membungkus dengan ArrayAccessNode atau RecordAccessNode sesuai token.
}

vector<ASTNode*> ASTConverter::convertIndexList(ParseNode* node) {

```

```

INPUT  : ParseNode <index-list>.
OUTPUT : Vector ASTNode indeks.
CARA KERJA :
1. Mengambil tiap elemen indeks (melewati/melompati comma).
2. Mengonversi menjadi VarNode/CharNode/NumberNode.
3. Mengumpulkan ke vector.
}

ASTNode* ASTConverter::convertExpression(ParseNode* node) {
    INPUT  : ParseNode <expression>.
    OUTPUT : ASTNode ekspresi (BinOpNode atau passthrough).
    CARA KERJA :
    1. Mengonversi <simple-expression> kiri.
    2. Jika ada operator relasional, mengonversi sisi kanan.
    3. Membentuk BinOpNode bila ada relasi, jika tidak mengembalikan
        sisi kiri.
}

ASTNode* ASTConverter::convertSimpleExpression(ParseNode* node) {
    INPUT  : ParseNode <simple-expression>.
    OUTPUT : ASTNode ekspresi sederhana.
    CARA KERJA :
    1. Menangani unary plus/minus jika ada.
    2. Mengonversi <term> pertama.
    3. Menggabungkan operator aditif berikutnya menjadi BinOpNode
        berantai.
}

ASTNode* ASTConverter::convertTerm(ParseNode* node) {
    INPUT  : ParseNode <term>.
    OUTPUT : ASTNode term.
    CARA KERJA :
    1. Mengonversi factor pertama.
    2. Menggabungkan operator multiplicative berikutnya menjadi
        BinOpNode berantai.
}

ASTNode* ASTConverter::convertFactor(ParseNode* node) {
    INPUT  : ParseNode <factor>.
    OUTPUT : ASTNode faktor.
    CARA KERJA :
    1. Mengecek apakah factor adalah call/variable, lalu konversi
        sesuai.
    2. Jika kurung, mengonversi ekspresi di dalamnya.
    3. Jika NOT, membuat UnaryOpNode.
    4. Jika literal, membuat node literal sesuai tipe token.
    5. Jika ident biasa, membuat VarNode.
}

```

Pada ASTConverter.cpp, implementasi disusun sebagai rangkaian fungsi konversi deterministik yang memetakan struktur Parse Tree sesuai grammar ke node-node AST yang lebih ringkas. Setiap fungsi mengekstrak token penting, mengonversi non-terminal ke node semantik, dan membangun struktur AST yang siap dipakai di tahap analisis berikutnya.

7. ASTNode.hpp & ASTNode.cpp

ASTNode.hpp

```
class ASTVisitor;

enum class ExprType {
    UNKNOWN, INTEGER, REAL, BOOLEAN, CHAR, STRING, VOID, ARRAY, RECORD,
    ENUM, SUBRANGE
};

// Base Class
class ASTNode {
public:
    ExprType exprType = ExprType::UNKNOWN;
    int tabIndex = -1; // Referensi indeks di symbol table (tab)
    int lev = -1;      // Kedalaman scope (0 = global)
    int lineNum = 0;   // Baris node ini berasal dari source code

    virtual ~ASTNode() = default;
    virtual void accept(ASTVisitor* visitor) = 0;
};

// STRUKTUR PROGRAM
class ProgramNode : public ASTNode {
public:
    string programName;
    vector<ASTNode*> declarations;
    ASTNode* mainBlock;

    ProgramNode(string name, vector<ASTNode*> decls, ASTNode* block);
    const string& getProgramName();
    void accept(ASTVisitor* visitor) override;
};

class CompoundNode : public ASTNode {
```

```

public:
    vector<ASTNode*> statements;

    CompoundNode(vector<ASTNode*> stmts);
    void accept(ASTVisitor* visitor) override;
};

// DEKLARASI
class ConstDeclNode : public ASTNode {
public:
    string constName;
    ASTNode* value;

    ConstDeclNode(string name, ASTNode* val);
    string getConstName();
    ASTNode* getValue();
    void accept(ASTVisitor* visitor) override;
};

class TypeDeclNode : public ASTNode {
public:
    string typeName;
    ASTNode* typeDef;

    TypeDeclNode(string name, ASTNode* def);
    string getTypeName();
    void accept(ASTVisitor* visitor) override;
};

class VarDeclNode : public ASTNode {
public:
    vector<string> varNames;
    ASTNode* typeDef;

    VarDeclNode(vector<string> names, ASTNode* def);
    vector<string> getVarNames();
    ASTNode* getTypeDef();
    void accept(ASTVisitor* visitor) override;
};

class ParamNode : public ASTNode {

```

```

public:
    vector<string> paramNames;
    ASTNode* typeDef;
    bool isVarParam; // True jika pass-by-reference (var parameter)

    ParamNode(vector<string> names, ASTNode* def, bool isVar);
    void accept(ASTVisitor* visitor) override;
};

class SubprogramDeclNode : public ASTNode {
public:
    bool isFunction;
    string subprogramName;
    vector<ASTNode*> parameters;
    ASTNode* returnType; // nullptr jika Procedure
    vector<ASTNode*> declarations;
    ASTNode* body;

    SubprogramDeclNode(bool isFunc, string name, vector<ASTNode*>
        params, ASTNode* retType, vector<ASTNode*> decls, ASTNode*
        bodyBlock);
    string getSubprogramName();
    ASTNode* getBody();
    void accept(ASTVisitor* visitor) override;
};

// DEFINISI TIPE
class NamedTypeNode : public ASTNode {
public:
    string typeName; // Misal: "integer", "Hari"

    NamedTypeNode(string name);
    void accept(ASTVisitor* visitor) override;
};

class ArrayTypeNode : public ASTNode {
public:
    ASTNode* indexType;
    ASTNode* elementType;

    ArrayTypeNode(ASTNode* idxType, ASTNode* elemType);

```



```

    void accept(ASTVisitor* visitor) override;
};

class RecordTypeNode : public ASTNode {
public:
    vector<ASTNode*> fields; // Isinya VarDeclNode

    RecordTypeNode(vector<ASTNode*> recFields);
    void accept(ASTVisitor* visitor) override;
};

class RangeNode : public ASTNode {
public:
    ASTNode* lowerBound;
    ASTNode* upperBound;

    RangeNode(ASTNode* low, ASTNode* high);
    void accept(ASTVisitor* visitor) override;
};

class EnumNode : public ASTNode {
public:
    vector<string> identifiers;

    EnumNode(vector<string> ids);
    void accept(ASTVisitor* visitor) override;
};

// STATEMENT
class AssignNode : public ASTNode {
public:
    ASTNode* target;
    ASTNode* value;

    AssignNode(ASTNode* tgt, ASTNode* val);
    void accept(ASTVisitor* visitor) override;
};

class IfNode : public ASTNode {
public:
    ASTNode* condition;

```

```

    ASTNode* thenBlock;
    ASTNode* elseBlock;

    IfNode(ASTNode* cond, ASTNode* tBlock, ASTNode* eBlock);
    void accept(ASTVisitor* visitor) override;
};

class CaseBlockNode : public ASTNode {
public:
    vector<ASTNode*> constants;
    ASTNode* statement;

    CaseBlockNode(vector<ASTNode*> consts, ASTNode* stmt);
    void accept(ASTVisitor* visitor) override;
};

class CaseNode : public ASTNode {
public:
    ASTNode* condition;
    vector<ASTNode*> cases; // Isinya CaseBlockNode

    CaseNode(ASTNode* cond, vector<ASTNode*> caseList);
    void accept(ASTVisitor* visitor) override;
};

class WhileNode : public ASTNode {
public:
    ASTNode* condition;
    ASTNode* loopBlock;

    WhileNode(ASTNode* cond, ASTNode* block);
    void accept(ASTVisitor* visitor) override;
};

class RepeatNode : public ASTNode {
public:
    vector<ASTNode*> statements;
    ASTNode* condition;

    RepeatNode(vector<ASTNode*> stmts, ASTNode* cond);
    void accept(ASTVisitor* visitor) override;
};

```

```

};

class ForNode : public ASTNode {
public:
    string iteratorName;
    ASTNode* startValue;
    ASTNode* endValue;
    bool isDownto;
    ASTNode* loopBlock;

    ForNode(string iterName, ASTNode* startVal, ASTNode* endVal, bool
downto, ASTNode* block);
    void accept(ASTVisitor* visitor) override;
};

class ProcCallNode : public ASTNode {
public:
    string procName;
    vector<ASTNode*> arguments;

    ProcCallNode(string name, vector<ASTNode*> args);
    vector<ASTNode*> getArguments();
    void accept(ASTVisitor* visitor) override;
};

// COMPONENT VARIABLE
class VarNode : public ASTNode {
public:
    string name;

    VarNode(string n);
    void accept(ASTVisitor* visitor) override;
};

class ArrayAccessNode : public ASTNode {
public:
    ASTNode* arrayVar;
    vector<ASTNode*> indices;

    ArrayAccessNode(ASTNode* arrVar, vector<ASTNode*> idxs);
    ASTNode* getArrayVar();

```

```

    void accept(ASTVisitor* visitor) override;
};

class RecordAccessNode : public ASTNode {
public:
    ASTNode* recordVar;
    string fieldName;

    RecordAccessNode(ASTNode* recVar, string fName);
    ASTNode* getRecordVar();
    void accept(ASTVisitor* visitor) override;
};

// EXPRESSION & FACTOR
class BinOpNode : public ASTNode {
public:
    string op;
    ASTNode* left;
    ASTNode* right;

    BinOpNode(string oper, ASTNode* l, ASTNode* r);
    void accept(ASTVisitor* visitor) override;
};

class UnaryOpNode : public ASTNode {
public:
    string op; // "+" "-" "not"
    ASTNode* operand;

    UnaryOpNode(string oper, ASTNode* expr);
    void accept(ASTVisitor* visitor) override;
};

class NumberNode : public ASTNode {
public:
    string value;
    bool isReal;

    NumberNode(string val, bool isR);
    void accept(ASTVisitor* visitor) override;
};

```

```

class StringNode : public ASTNode {
public:
    string value;

    StringNode(string val);
    void accept(ASTVisitor* visitor) override;
};

class CharNode : public ASTNode {
public:
    string value;

    CharNode(string val);
    void accept(ASTVisitor* visitor) override;
};

```

ASTNode.cpp

```

ProgramNode::ProgramNode(string name, vector<ASTNode*> decls, ASTNode*
block) {
    INPUT : Nama program, daftar deklarasi, dan blok utama program.
    OUTPUT : ProgramNode.
    CARA KERJA :
    1. Mengisi programName dengan nama program.
    2. Mengisi declarations dengan daftar deklarasi global program.
    3. Mengisi mainBlock dengan node blok utama program.
}

void ProgramNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitProgramNode pada visitor.
    3. Mengirim this agar visitor memproses ProgramNode saat ini.
}

const string& ProgramNode::getProgramName() {
    INPUT : -
    OUTPUT : Nama program.
    CARA KERJA :
    1. Mengambil nilai programName dari ProgramNode.
    2. Mengembalikan nama program sebagai const reference agar string

```

tidak disalin.

}

CompoundNode::CompoundNode(vector<ASTNode*> stmts) {

INPUT : Daftar statement.

OUTPUT : CompoundNode.

CARA KERJA :

1. Menerima kumpulan node statement.

2. Menyimpan kumpulan statement tersebut ke dalam statements.

}

void CompoundNode::accept(ASTVisitor* visitor) {

INPUT : ASTVisitor.

OUTPUT : -

CARA KERJA :

1. Menerima object visitor.

2. Memanggil visitCompoundNode pada visitor.

3. Mengirim this agar visitor memproses CompoundNode saat ini.

}

ConstDeclNode::ConstDeclNode(string name, ASTNode* val) {

INPUT : Nama konstanta dan node nilai konstanta.

OUTPUT : ConstDeclNode.

CARA KERJA :

1. Mengisi constName dengan identifier konstanta.

2. Mengisi value dengan node literal atau ekspresi yang menjadi nilai konstanta.

}

void ConstDeclNode::accept(ASTVisitor* visitor) {

INPUT : ASTVisitor.

OUTPUT : -

CARA KERJA :

1. Menerima object visitor.

2. Memanggil visitConstDeclNode pada visitor.

3. Mengirim this agar visitor memproses ConstDeclNode saat ini.

}

string ConstDeclNode::getConstName() {

INPUT : -

OUTPUT : Nama konstanta.

CARA KERJA :

1. Mengambil identifier konstanta dari constName.

2. Mengembalikan nama konstanta tersebut.

}

```

ASTNode* ConstDeclNode::getValue() {
    INPUT : -
    OUTPUT : Node nilai konstanta.
    CARA KERJA :
    1. Mengambil pointer value dari ConstDeclNode.
    2. Mengembalikan node literal atau ekspresi yang menjadi isi
    konstanta.
}

TypeDeclNode::TypeDeclNode(string name, ASTNode* def) {
    INPUT : Nama tipe dan node definisi tipe.
    OUTPUT : TypeDeclNode.
    CARA KERJA :
    1. Mengisi typeName dengan nama tipe yang dideklarasikan.
    2. Mengisi typeDef dengan node yang menjelaskan bentuk tipe
    tersebut.
}

void TypeDeclNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitTypeDeclNode pada visitor.
    3. Mengirim this agar visitor memproses TypeDeclNode saat ini.
}

string TypeDeclNode::getTypeName() {
    INPUT : -
    OUTPUT : Nama tipe.
    CARA KERJA :
    1. Mengambil identifier tipe dari typeName.
    2. Mengembalikan nama tipe yang dideklarasikan.
}

VarDeclNode::VarDeclNode(vector<string> names, ASTNode* def) {
    INPUT : Daftar nama variabel dan node definisi tipe.
    OUTPUT : VarDeclNode.
    CARA KERJA :
    1. Mengisi varNames dengan semua identifier variabel dalam satu
    deklarasi.
    2. Mengisi typeDef dengan node tipe yang berlaku untuk semua
    variabel tersebut.
}

void VarDeclNode::accept(ASTVisitor* visitor) {

```

```

    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitVarDeclNode pada visitor.
    3. Mengirim this agar visitor memproses VarDeclNode saat ini.
}

vector<string> VarDeclNode::getVarNames() {
    INPUT : -
    OUTPUT : Daftar nama variabel.
    CARA KERJA :
    1. Mengambil vector varNames dari VarDeclNode.
    2. Mengembalikan daftar identifier variabel yang dideklarasikan.
}

ASTNode* VarDeclNode::getTypeDef() {
    INPUT : -
    OUTPUT : Node definisi tipe variabel.
    CARA KERJA :
    1. Mengambil pointer typeDef dari VarDeclNode.
    2. Mengembalikan node yang menjelaskan tipe variabel, seperti
    NamedTypeNode, ArrayTypeNode, RecordTypeNode, RangeNode, atau
    EnumNode.
}

ParamNode::ParamNode(vector<string> names, ASTNode* def, bool isVar) {
    INPUT : Daftar nama parameter, node tipe parameter, dan status var
    parameter.
    OUTPUT : ParamNode.
    CARA KERJA :
    1. Mengisi paramNames dengan daftar identifier parameter.
    2. Mengisi typeDef dengan node tipe parameter.
    3. Mengisi isVarParam dengan true jika parameter memakai keyword
    var, atau false jika tidak.
}

void ParamNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitParamNode pada visitor.
    3. Mengirim this agar visitor memproses ParamNode saat ini.
}

```



```

SubprogramDeclNode::SubprogramDeclNode(bool isFunc, string name,
vector<ASTNode*> params, ASTNode* retType, vector<ASTNode*> decls,
ASTNode* bodyBlock) {
    INPUT : Status function/procedure, nama subprogram, daftar
parameter, tipe return, deklarasi lokal, dan body.
    OUTPUT : SubprogramDeclNode.
    CARA KERJA :
    1. Mengisi isFunction dengan status apakah subprogram adalah
function.
    2. Mengisi subprogramName dengan nama procedure atau function.
    3. Mengisi parameters dengan daftar parameter subprogram.
    4. Mengisi returnType dengan tipe hasil function, atau nullptr jika
subprogram adalah procedure.
    5. Mengisi declarations dengan deklarasi lokal subprogram.
    6. Mengisi body dengan blok statement isi subprogram.
}

void SubprogramDeclNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitSubprogramDeclNode pada visitor.
    3. Mengirim this agar visitor memproses SubprogramDeclNode saat
ini.
}

string SubprogramDeclNode::getSubprogramName() {
    INPUT : -
    OUTPUT : Nama subprogram.
    CARA KERJA :
    1. Mengambil identifier subprogram dari subprogramName.
    2. Mengembalikan nama procedure atau function tersebut.
}

ASTNode* SubprogramDeclNode::getBody() {
    INPUT : -
    OUTPUT : Node body subprogram.
    CARA KERJA :
    1. Mengambil pointer body dari SubprogramDeclNode.
    2. Mengembalikan blok statement yang menjadi isi procedure atau
function.
}

NamedTypeNode::NamedTypeNode(string name) {
    INPUT : Nama tipe.
    OUTPUT : NamedTypeNode.
}

```

```

    CARA KERJA :
    1. Menerima nama tipe dari hasil parsing.
    2. Menyimpan nama tipe tersebut ke dalam typeName.
}

void NamedTypeNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitNamedTypeNode pada visitor.
    3. Mengirim this agar visitor memproses NamedTypeNode saat ini.
}

ArrayTypeNode::ArrayTypeNode(ASTNode* idxType, ASTNode* elemType) {
    INPUT : Node tipe indeks dan node tipe elemen array.
    OUTPUT : ArrayTypeNode.
    CARA KERJA :
    1. Mengisi idxType dengan node yang menjelaskan indeks array.
    2. Mengisi elementType dengan node yang menjelaskan tipe data
    elemen array.
}

void ArrayTypeNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitArrayTypeNode pada visitor.
    3. Mengirim this agar visitor memproses ArrayTypeNode saat ini.
}

RecordTypeNode::RecordTypeNode(vector<ASTNode*> recFields) {
    INPUT : Daftar field record.
    OUTPUT : RecordTypeNode.
    CARA KERJA :
    1. Menerima kumpulan node field record.
    2. Menyimpan kumpulan field tersebut ke dalam fields.
}

void RecordTypeNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitRecordTypeNode pada visitor.
}

```

```
    3. Mengirim this agar visitor memproses RecordTypeNode saat ini.  
}
```

```
RangeNode::RangeNode(ASTNode* low, ASTNode* high) {  
    INPUT : Node batas bawah dan node batas atas subrange.  
    OUTPUT : RangeNode.  
    CARA KERJA :  
    1. Mengisi lowerBound dengan node batas awal subrange.  
    2. Mengisi upperBound dengan node batas akhir subrange.  
}
```

```
void RangeNode::accept(ASTVisitor* visitor) {  
    INPUT : ASTVisitor.  
    OUTPUT : -  
    CARA KERJA :  
    1. Menerima object visitor.  
    2. Memanggil visitRangeNode pada visitor.  
    3. Mengirim this agar visitor memproses RangeNode saat ini.  
}
```

```
EnumNode::EnumNode(vector<string> ids) {  
    INPUT : Daftar identifier enum.  
    OUTPUT : EnumNode.  
    CARA KERJA :  
    1. Menerima semua identifier yang berada di dalam tipe enumerasi.  
    2. Menyimpan daftar identifier tersebut ke dalam identifiers.  
}
```

```
void EnumNode::accept(ASTVisitor* visitor) {  
    INPUT : ASTVisitor.  
    OUTPUT : -  
    CARA KERJA :  
    1. Menerima object visitor.  
    2. Memanggil visitEnumNode pada visitor.  
    3. Mengirim this agar visitor memproses EnumNode saat ini.  
}
```

```
AssignNode::AssignNode(ASTNode* tgt, ASTNode* val) {  
    INPUT : Node target assignment dan node ekspresi nilai.  
    OUTPUT : AssignNode.  
    CARA KERJA :  
    1. Mengisi target dengan variabel atau lokasi yang akan diberi nilai.  
    2. Mengisi value dengan ekspresi yang hasilnya akan disimpan ke target.
```

```

}

void AssignNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitAssignNode pada visitor.
    3. Mengirim this agar visitor memproses AssignNode saat ini.
}

IfNode::IfNode(ASTNode* cond, ASTNode* tBlock, ASTNode* eBlock) {
    INPUT : Node kondisi, blok then, dan blok else.
    OUTPUT : IfNode.
    CARA KERJA :
    1. Mengisi condition dengan ekspresi kondisi if.
    2. Mengisi thenBlock dengan blok yang dijalankan jika kondisi benar.
    3. Mengisi elseBlock dengan blok yang dijalankan jika kondisi salah.
}

void IfNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitIfNode pada visitor.
    3. Mengirim this agar visitor memproses IfNode saat ini.
}

CaseBlockNode::CaseBlockNode(vector<ASTNode*> consts, ASTNode* stmt) {
    INPUT : Daftar konstanta case dan statement.
    OUTPUT : CaseBlockNode.
    CARA KERJA :
    1. Mengisi constants dengan label-label konstanta pada satu cabang case.
    2. Mengisi statement dengan perintah yang dijalankan jika label case cocok.
}

void CaseBlockNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.

```

```

    2. Memanggil visitCaseBlockNode pada visitor.
    3. Mengirim this agar visitor memproses CaseBlockNode saat ini.
}

CaseNode::CaseNode(ASTNode* cond, vector<ASTNode*> caseList) {
    INPUT : Node ekspresi case dan daftar cabang case.
    OUTPUT : CaseNode.
    CARA KERJA :
    1. Mengisi condition dengan ekspresi yang akan dibandingkan.
    2. Mengisi cases dengan daftar CaseBlockNode yang menjadi
cabang-cabang case.
}

void CaseNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitCaseNode pada visitor.
    3. Mengirim this agar visitor memproses CaseNode saat ini.
}

WhileNode::WhileNode(ASTNode* cond, ASTNode* block) {
    INPUT : Node kondisi dan blok loop.
    OUTPUT : WhileNode.
    CARA KERJA :
    1. Mengisi condition dengan ekspresi kondisi while.
    2. Mengisi loopBlock dengan blok statement yang diulang selama
kondisi benar.
}

void WhileNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitWhileNode pada visitor.
    3. Mengirim this agar visitor memproses WhileNode saat ini.
}

RepeatNode::RepeatNode(vector<ASTNode*> stmts, ASTNode* cond) {
    INPUT : Daftar statement dan node kondisi until.
    OUTPUT : RepeatNode.
    CARA KERJA :
    1. Mengisi statements dengan daftar statement yang akan dijalankan

```

berulang.

2. Mengisi condition dengan ekspresi penghenti repeat-until.

}

```
void RepeatNode::accept(ASTVisitor* visitor) {
```

INPUT : ASTVisitor.

OUTPUT : -

CARA KERJA :

1. Menerima object visitor.
2. Memanggil visitRepeatNode pada visitor.
3. Mengirim this agar visitor memproses RepeatNode saat ini.

}

```
ForNode::ForNode(string iterName, ASTNode* startVal, ASTNode* endVal,  
bool downto, ASTNode* block) {
```

INPUT : Nama iterator, nilai awal, nilai akhir, status downto, dan blok loop.

OUTPUT : ForNode.

CARA KERJA :

1. Mengisi iteratorName dengan nama variabel pengendali loop.
2. Mengisi startValue dengan ekspresi nilai awal loop.
3. Mengisi endValue dengan ekspresi nilai akhir loop.
4. Mengisi isDownto dengan arah iterasi, true untuk downto dan false untuk to.
5. Mengisi loopBlock dengan statement yang dijalankan pada setiap iterasi.

}

```
void ForNode::accept(ASTVisitor* visitor) {
```

INPUT : ASTVisitor.

OUTPUT : -

CARA KERJA :

1. Menerima object visitor.
2. Memanggil visitForNode pada visitor.
3. Mengirim this agar visitor memproses ForNode saat ini.

}

```
ProcCallNode::ProcCallNode(string name, vector<ASTNode*> args) {
```

INPUT : Nama procedure/function dan daftar argumen.

OUTPUT : ProcCallNode.

CARA KERJA :

1. Mengisi procName dengan nama subprogram yang dipanggil.
2. Mengisi arguments dengan ekspresi argumen yang dikirim pada pemanggilan.

}

```

void ProcCallNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitProcCallNode pada visitor.
    3. Mengirim this agar visitor memproses ProcCallNode saat ini.
}

vector<ASTNode*> ProcCallNode::getArguments() {
    INPUT : -
    OUTPUT : Daftar node argumen pemanggilan.
    CARA KERJA :
    1. Mengambil daftar argumen dari arguments.
    2. Mengembalikan kumpulan ekspresi yang dikirim ke procedure atau
function.
}

VarNode::VarNode(string n) {
    INPUT : Nama variabel.
    OUTPUT : VarNode.
    CARA KERJA :
    1. Menerima identifier variabel dari hasil parsing.
    2. Menyimpan identifier tersebut ke dalam name.
}

void VarNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitVarNode pada visitor.
    3. Mengirim this agar visitor memproses VarNode saat ini.
}

ArrayAccessNode::ArrayAccessNode(ASTNode* arrVar, vector<ASTNode*>
idxs) {
    INPUT : Node array dan daftar indeks.
    OUTPUT : ArrayAccessNode.
    CARA KERJA :
    1. Mengisi arrayVar dengan node array yang diakses.
    2. Mengisi indices dengan daftar ekspresi indeks yang menentukan
elemen array.
}

void ArrayAccessNode::accept(ASTVisitor* visitor) {

```

```

    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitArrayAccessNode pada visitor.
    3. Mengirim this agar visitor memproses ArrayAccessNode saat ini.
}

ASTNode* ArrayAccessNode::getArrayVar() {
    INPUT : -
    OUTPUT : Node array yang sedang diakses.
    CARA KERJA :
    1. Mengambil pointer arrayVar dari ArrayAccessNode.
    2. Mengembalikan node array sebelum indeks diterapkan.
}

RecordAccessNode::RecordAccessNode(ASTNode* recVar, string fName) {
    INPUT : Node record dan nama field.
    OUTPUT : RecordAccessNode.
    CARA KERJA :
    1. Mengisi recordVar dengan node record yang diakses.
    2. Mengisi fieldName dengan nama field yang dipilih dari record
    tersebut.
}

void RecordAccessNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitRecordAccessNode pada visitor.
    3. Mengirim this agar visitor memproses RecordAccessNode saat ini.
}

ASTNode* RecordAccessNode::getRecordVar() {
    INPUT : -
    OUTPUT : Node record yang sedang diakses.
    CARA KERJA :
    1. Mengambil pointer recordVar dari RecordAccessNode.
    2. Mengembalikan node record sebelum field dipilih.
}

BinOpNode::BinOpNode(string oper, ASTNode* l, ASTNode* r) {
    INPUT : Operator biner, operand kiri, dan operand kanan.
    OUTPUT : BinOpNode.
    CARA KERJA :

```



```

    1. Mengisi op dengan simbol operator biner.
    2. Mengisi left dengan ekspresi operand kiri.
    3. Mengisi right dengan ekspresi operand kanan.
}

void BinOpNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitBinOpNode pada visitor.
    3. Mengirim this agar visitor memproses BinOpNode saat ini.
}

UnaryOpNode::UnaryOpNode(string oper, ASTNode* expr) {
    INPUT : Operator unary dan operand.
    OUTPUT : UnaryOpNode.
    CARA KERJA :
    1. Mengisi op dengan simbol operator unary seperti +, -, atau not.
    2. Mengisi operand dengan ekspresi yang dikenai operator unary
    tersebut.
}

void UnaryOpNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitUnaryOpNode pada visitor.
    3. Mengirim this agar visitor memproses UnaryOpNode saat ini.
}

NumberNode::NumberNode(string val, bool isR) {
    INPUT : Teks literal angka dan status real.
    OUTPUT : NumberNode.
    CARA KERJA :
    1. Mengisi value dengan teks angka yang muncul di source code.
    2. Mengisi isReal dengan true jika angka adalah real, atau false
    jika angka adalah integer.
}

void NumberNode::accept(ASTVisitor* visitor) {
    INPUT : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.

```

```

    2. Memanggil visitNumberNode pada visitor.
    3. Mengirim this agar visitor memproses NumberNode saat ini.
}

StringNode::StringNode(string val) {
    INPUT  : Teks literal string.
    OUTPUT : StringNode.
    CARA KERJA :
    1. Menerima isi literal string dari hasil parsing.
    2. Menyimpan isi literal string tersebut ke dalam value.
}

void StringNode::accept(ASTVisitor* visitor) {
    INPUT  : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitStringNode pada visitor.
    3. Mengirim this agar visitor memproses StringNode saat ini.
}

CharNode::CharNode(string val) {
    INPUT  : Teks literal char.
    OUTPUT : CharNode.
    CARA KERJA :
    1. Menerima isi literal karakter dari hasil parsing.
    2. Menyimpan isi literal karakter tersebut ke dalam value.
}

void CharNode::accept(ASTVisitor* visitor) {
    INPUT  : ASTVisitor.
    OUTPUT : -
    CARA KERJA :
    1. Menerima object visitor.
    2. Memanggil visitCharNode pada visitor.
    3. Mengirim this agar visitor memproses CharNode saat ini.
}

```

2.3. Aturan Syntax-Directed Translation Scheme (SDT) pembentukan AST

No	Production Rule	Semantic Rule
----	-----------------	---------------

1	<code><program> → <program-header> <declaration-part> <compound-statement> '.'</code>	return new ProgramNode(header_ident.getToken().value, declarations_vector, mainBlock_node)
2	<code><program-header> → programsy ident ';' </code>	(Mengembalikan pointer ke node ident untuk diekstrak namanya)
3	<code><declaration-part> → (<const-decl>)* (<type-decl>)* (<var-decl>)* (<subprog-decl>)*`</code>	allDecls.insert(const_vector, type_vector, var_vector) allDecls.push_back(subprogram_node) return allDecls
4	<code><const-declaration> → constsy (ident '=' <constant> ';') +</code>	result.push_back(new ConstDeclNode(ident.getToken().value, constant_node))
5	<code><type-declaration> → typesy (ident '=' <type> ';') +</code>	result.push_back(new TypeDeclNode(ident.getToken().value, type_node))
6	<code><var-declaration> → varsy (<identifier-list> ':' <type> ';') +</code>	result.push_back(new VarDeclNode(identifier_vector, type_node))
7	<code><identifier-list> → ident (',' ident)*</code>	names.push_back(ident.getToken().value) return names
8	<code><subprogram-declaration> → <procedure-decl> <function-decl></code>	(Bergantung pada rule anak)
9	<code><procedure-decl> → proceduresy ident ['(' <formal-parameter-list> ')'] '; ' <block> ';'</code>	return new SubprogramDeclNode(false, ident.getToken().value, params_vector, nullptr, localDecls_vector, bodyBlock_node)
10	<code><function-decl> → functionsy ident ['(' <formal-parameter-list> ')'] ' : ' ident ';' <block> ';'</code>	return new SubprogramDeclNode(true, ident.getToken().value, params_vector, new NamedTypeNode(ident.getToken().value), localDecls_vector, bodyBlock_node)
11	<code><formal-parameter-list> → <parameter-group> (';' <parameter-group>)*</code>	params.push_back(parameterGroup_node)

12	<parameter-group> → <identifier-list> ':' (ident <array-type>)	return new ParamNode(identifier_vector, type_node, false)
13	<type> → ident <array-type> <range> <enumerated> <record-type>	return new NamedTypeNode(ident.getToken().value)
14	<array-type> → arraysy '[' (<range> ident) ']' ofsy <type>	return new ArrayTypeNode(range_node, type_node)
15	<range> → <constant> '..' <constant>	return new RangeNode(lowerBound_node, upperBound_node)
16	<enumerated> → '(' ident (',' ident)* ')'	identifiers.push_back(ident.getToken() .value) return new EnumNode(identifiers)
17	<record-type> → recordsy <field-list> endsy	return new RecordTypeNode(fields_vector)
18	<field-part> → <identifier-list> ':' <type>	fields.push_back(new VarDeclNode(identifier_vector, type_node))
19	<compound-statement> → beginsy <statement-list> endsy	return new CompoundNode(statements_vector)
20	<statement-list> → <statement> ('; <statement>)*	stmts.push_back(statement_node)
21	<assignment-statement> → <variable> ':=' <expression>	return new AssignNode(variable_node, expression_node)
22	<if-statement> → ifsy <expression> then <statement> [elsy <statement>]	return new IfNode(condition_node, thenBlock_node, elseBlock_node)
23	<case-statement> → casesy <expression> ofsy <case-block> endsy	return new CaseNode(condition_node, caseBlocks_vector)
24	<case-block> → <constant> (',' <constant>)* ':'	return new CaseBlockNode(constants_vector, statement_node)

	<statement> (';' <case-block>)*	
25	<while-statement> → whilesy <expression> dosy <compound-statement>	return new WhileNode(condition_node, loopBlock_node)
26	<repeat-statement> → repeatsy <statement-list> untilsy <expression>	return new RepeatNode(statements_vector, condition_node)
27	<for-statement> → forsy ident ':= ' <expression> (tosy downtosy) <expression> dosy <compound-statement>	return new ForNode(ident.getToken().value, startVal_node, endVal_node, isDownto_bool, loopBlock_node)
28	<procedure-call> → ident ['(' <parameter-list> ')']	return new ProcCallNode(ident.getToken().value, args_vector)
29	<variable> → ident (<component-variable>)*	currentVar = new VarNode(ident.getToken().value) currentVar = new ArrayAccessNode(currentVar, indices_vector) (jika array) currentVar = new RecordAccessNode(currentVar, field_ident.getToken().value) (jika record) return currentVar
30	<expression> → <simple-expression> [<relational-operator> <simple-expression>]	return new BinOpNode(relationalOp.getToken().value, left_node, right_node)
31	<simple-expression> → ['+' '-'] <term> (<additive-operator> <term>)*	currentLeft = new UnaryOpNode(sign.getToken().value, term_node) (jika unary) currentLeft = new BinOpNode(additiveOp.getToken().value, currentLeft, nextRight)
32	<term> → <factor> (<multiplicative-operator> <factor>)*	currentLeft = new BinOpNode(multiplicativeOp.getToken().value, currentLeft, nextRight)
33	<factor> → notsy <factor>	return new UnaryOpNode("not", factor_node)

34	<code><factor> → ident intcon realcon charcon string</code>	<code>return new VarNode(valStr) atau new NumberNode(valStr, isReal) atau new CharNode(valStr) atau new StringNode(valStr)`</code>
----	---	--

Tabel 2.3 Aturan SDT pembentukan AST

Keterangan:

- **[...] = Opsional** (Muncul 0 atau 1 kali).
- **(...)* = Kleen Star / Repetisi Bebas** (Muncul 0, 1, 2, hingga n kali).
- **(...)+ = Kleen Plus / Repetisi Wajib** (Muncul minimal 1 kali hingga n kali).

2.4. Predefined Identifier

Identifier	Keterangan
real	Memiliki type Real e.g. someArray[1..10] of Real
integer	Memiliki type Integer e.g. procedure print(x: integer);
char	Memiliki type Char e.g. var a: Char
boolean	Memiliki type Boolean e.g. var b: Boolean
string	Memiliki type String e.g. var c: String
true	Memiliki type Boolean dengan nilai true e.g. const isTrue == true
false	Memiliki type Boolean dengan nilai false e.g. if (someVar = false) then
writeln	Prosedur bawaan untuk mencetak output ke layar beserta newline. e.g. writeln('Hello World');
write	Prosedur bawaan untuk mencetak output ke layar tanpa ganti baris. e.g. write('Masukkan nama: ');

readln	Prosedur bawaan untuk menerima masukan data pengguna beserta newline. e.g. readln(nama);
read	Prosedur bawaan untuk menerima masukan data pengguna. e.g. read(x);
abs	Fungsi bawaan bertipe Integer/Real untuk menghitung nilai mutlak. e.g. jarak := abs(-10);
sqrt	Fungsi bawaan bertipe Real untuk menghitung akar kuadrat. e.g. akar := sqrt(16.0);
length	Fungsi bawaan bertipe Integer untuk menghitung panjang data/string. e.g. panjang := length('Arion');

Tabel 2.4 Predefined Identifier

2.5. Type Compatibility Rule

1. Tipe Dasar

No	Nama Type	Keterangan
1	Real	realcon memiliki type Real.
2	Integer	intcon memiliki type Integer.
3	Char	charcon memiliki type Char.
4	Boolean	true dan false memiliki type Boolean. term dan expression yang diproduksi dari notsy, andsy, dan orsy memiliki type Boolean. Expression yang diproduksi dari relational operator juga memiliki type Boolean.
5	String	string memiliki type String. Jika string memiliki panjang eksplisit seperti String[n], panjang tersebut menjadi bagian dari informasi type.

6	Subrange	Type dari node Range ditentukan oleh type constant di dalamnya. Constant kiri dan kanan harus compatible. Lower bound tidak boleh lebih besar daripada upper bound. Subrange tidak boleh memiliki type Real.
7	Enumerated	Type dari node Enumerated ditentukan oleh domain enumerasi yang dibentuk dari identifier di dalamnya. Identifier dalam satu enumerated tidak boleh duplikat. Setiap enumerated memiliki enumDomainId untuk membedakan domain enum yang berbeda.

Tabel 2.5.1 Tipe Dasar

2. Structured Type

No	Nama Type	Keterangan
1	Array	Bentuk array adalah array [T1] of T2. T1 adalah index type dan harus berupa simple type, tetapi tidak boleh Real. Char boleh menjadi index type. T2 adalah element type dan boleh berupa type apa pun.
2	Record	Bentuk record adalah record field: T; end. Setiap field di dalam record boleh memiliki type apa pun. Akses field hanya valid jika nama field terdapat di dalam record tersebut.
3	Named Structured Type	Named structured type didefinisikan melalui type-declaration. Compatibility tidak lagi ditentukan hanya dari nama type, tetapi dari struktur type.
4	Anonymous Structured Type	Anonymous structured type didefinisikan langsung di deklarasi variable. Compatibility tidak lagi otomatis ditolak hanya karena anonymous, tetapi ditentukan dari struktur record atau array.

Tabel 2.5.2 Structured Type

3. Rule untuk Expression dan Statement

No	Konteks	Keterangan
----	---------	------------

1	if condition then ...	Expression pada condition harus bertipe Boolean atau Integer. Jika Integer, nilai 0 dianggap false, dan nilai non-zero dianggap true.
2	while condition do ...	Expression pada condition harus bertipe Boolean atau Integer. Jika Integer, nilai 0 dianggap false, dan nilai non-zero dianggap true.
3	repeat ... until condition	Expression pada condition harus bertipe Boolean atau Integer. Jika Integer, nilai 0 dianggap false, dan nilai non-zero dianggap true.
4	Unary + dan -	Operand harus bertipe numerik, yaitu Integer atau Real. Hasil operasi mengikuti type operand.
5	Unary not	Operand harus bertipe Boolean. Hasil operasi bertipe Boolean.
6	Binary arithmetic +, -, *	Kedua operand harus bertipe numerik. Jika salah satu operand bertipe Real, hasilnya bertipe Real. Jika keduanya bertipe Integer, hasilnya bertipe Integer.
7	Binary arithmetic /	Kedua operand harus bertipe numerik. Hasil operasi selalu bertipe Real.
8	Binary arithmetic div, mod	Kedua operand harus bertipe Integer. Hasil operasi bertipe Integer.
9	Binary logical and, or	Kedua operand harus bertipe Boolean. Hasil operasi bertipe Boolean.
10	Relational operator	Kedua operand harus comparable. Jika valid, hasil expression bertipe Boolean. Char dan String dapat dibandingkan dengan urutan ASCII/lexicographic. Enum hanya comparable jika berasal dari domain enum yang sama.
11	Assignment statement	Type expression di kanan harus assignment-compatible dengan type identifier di kiri.

12	Range node	Constant kiri dan constant kanan harus memiliki type yang compatible. Lower bound tidak boleh lebih besar daripada upper bound. Type Real tidak boleh digunakan sebagai subrange.
13	Array index access	Expression indeks harus compatible dengan index type array.
14	Record field access	Field hanya valid jika nama field terdapat di dalam record.

Tabel 2.5.3 Rule untuk Expression dan Statement

4. Compatibility Type

No	Kondisi	Compatibility	Keterangan
1	T1 dan T2 memiliki type yang sama	Ya	Contoh: Integer dengan Integer, Boolean dengan Boolean, atau Char dengan Char.
2	Integer dan Real	Ya	Compatible untuk operasi numerik dan assignment. Integer dapat dipromosikan menjadi Real, sedangkan Real dapat dikonversi menjadi Integer dengan truncation pada assignment.
3	Salah satu type adalah subrange dari base type lainnya	Ya	Contoh: 1..10 compatible dengan Integer.
4	Kedua type adalah subrange dari base type yang sama	Ya	Contoh: 1..10 compatible dengan 50..100 karena sama-sama subrange dari Integer.
5	Kedua type adalah String dengan panjang yang sama	Ya	String[n] compatible dengan String[n].

6	Kedua type adalah String dengan panjang berbeda	Tidak	String[n] tidak compatible dengan String[m] jika $n \neq m$.
7	Real dengan subrange berbasis Real	Tidak	Subrange tidak boleh berbasis Real.
8	Dua array	Ya, jika struktur compatible	Array compatible jika index type compatible dan element type compatible.
9	Dua record	Ya, jika struktur compatible	Record compatible jika jumlah field, nama field, urutan field, dan type field compatible.
10	Dua enum dari domain yang sama	Ya	Compatible jika keduanya memiliki enumDomainId yang sama.
11	Dua enum dari domain berbeda	Tidak	Incompatible walaupun identifier tampak mirip atau sama-sama bertipe Enum.
12	Structured type dengan struktur berbeda	Tidak	Array atau record tidak compatible jika struktur type-nya berbeda.

Tabel 2.5.4 Compatibility Rule

5. Assignment Compatibility

No	Target Type T1	Source Type T2	Assignment Compatibility	Keterangan
1	Real	Integer	Ya	Integer dipromosikan menjadi Real.
2	Integer	Real	Ya	Real dikonversi menjadi Integer dengan truncation. Contoh: 3.9 menjadi 3.

3	T1	T2 dengan type sama	Ya	Berlaku untuk type dasar seperti Integer, Boolean, Char, String, dan type lain yang compatible.
4	Subrange	Nilai dari base type yang berada dalam range	Ya	Contoh: variable bertipe 1..10 dapat menerima nilai 5.
5	Subrange	Nilai dari base type yang berada di luar range	Tidak	Contoh: variable bertipe 1..10 tidak dapat menerima nilai 11.
6	Base type	Subrange dari base type yang sama	Ya	Contoh: Integer dapat menerima nilai dari subrange 1..10.
7	String[n]	String[n]	Ya	Assignment string valid jika keduanya compatible dan memiliki panjang yang sama.
8	String[n]	String[m], dengan $n \neq m$	Tidak	Panjang string berbeda dianggap assignment-incompatible.
9	Enum	Enum dari domain yang sama	Ya	Assignment valid jika kedua enum memiliki enumDomainId yang sama.
10	Enum	Enum dari domain berbeda	Tidak	Assignment invalid walaupun keduanya sama-sama bertipe Enum.

11	Record	Record dengan struktur compatible	Ya	Assignment record valid jika field-field record compatible.
12	Record	Record dengan struktur berbeda	Tidak	Assignment record invalid jika jumlah, nama, urutan, atau type field berbeda.
13	Array	Array dengan struktur compatible	Ya	Assignment array valid jika index type dan element type compatible.
14	Array	Array dengan struktur berbeda	Tidak	Assignment array invalid jika index type atau element type tidak compatible.

Tabel 2.5.5 Assignment Compatibility

BAB III

PENGUJIAN

3.1 Test Case 1

Test Case berhasil karena semua deklarasi, assignment, akses array, akses record, enum, case, dan for sudah valid secara semantik.

INPUT

```

1  program testCase1;
2
3  const
4      PI = 3.14;
5      MAX = 100;
6      HURUF = 'A';
7      PESAN = 'Halo Dunia';
8
9  type
10     Hari = (Senin, Selasa, Rabu, Kamis, Jumat);
11     Batas = 1 .. 100;
12     Siswa = record
13         umur : integer;
14         nilai : real;
15         lulus : boolean
16     end;
17     DataSiswa = array [1..100] of Siswa;
18     Matriks = array [1..10] of array [1..10] of integer;
19
20  var
21     i, j, k : integer;
22     murid : Siswa;
23     kelas : DataSiswa;
24     papan : Matriks;
25     libur : Hari;
26
27  begin
28     i := +10;
29     j := -5;
30
31     murid.umur := 15;
32     murid.nilai := PI * 2.0;
33     kelas[1].lulus := true;
34     papan[1][2] := 100;
35     libur := Senin;
36
37     case i of
38         1, 2, 3 : k := 10;
39         4 : k := 20;
40         5 : k := 0
41     end;
42
43     for i := 1 to MAX do
44     begin
45         if i = 50 then
46             kelas[i].lulus := true
47         else
48             kelas[i].lulus := false;
49         end;
50     end.

```

OUTPUT

```
1 SEMANTIC ERROR REPORT
2 NO SEMANTIC ERRORS (Program is semantically valid)
3
4 TAB (hanya sebagian yang relevan):
5 idx id obj type ref nrm lev adr link
6 -----
7 33 testCase1 program void 33 1 0 1 0
8 34 PI constant real 34 1 0 4 0
9 35 MAX constant integer 35 1 0 5 0
10 36 HURUF constant char 36 1 0 6 0
11 37 PESAN constant string 37 1 0 7 0
12 38 Senin constant enum 38 1 0 10 0
13 39 Selasa constant enum 39 1 0 10 0
14 40 Rabu constant enum 40 1 0 10 0
15 41 Kamis constant enum 41 1 0 10 0
16 42 Jumat constant enum 42 1 0 10 0
17 43 Hari type enum 43 1 0 10 0
18 44 Batas type subrange 44 1 0 11 0
19 45 umur variable integer 45 1 0 14 0
20 46 nilai variable real 46 1 0 11 0
21 47 lulus variable boolean 47 1 0 0 0
22 48 Siswa type record 48 1 0 12 0
23 49 DataSiswa type array 49 1 0 17 0
24 50 Matriks type array 50 1 0 18 0
25 51 i variable integer 51 1 0 0 0
26 52 j variable integer 52 1 0 0 0
27 53 k variable integer 53 1 0 0 0
28 54 murid variable record 54 1 0 0 0
29 55 kelas variable array 55 1 0 0 0
30 56 papan variable array 56 1 0 0 0
31 57 libur variable enum 57 1 0 0 0
32
33 btab:
34 idx last lpar psze vsze
35 -----
36 0 57 0 0 10
37
38 ATAB:
39 | (No arrays declared)
40
```


DECORATED AST:

ProgramNode('testCase1') → block_index:0, lev:0

```

└─ Declarations
  └─ ConstDecl('PI') → tab_index:34, type:real, lev:0
  └─ ConstDecl('MAX') → tab_index:35, type:integer, lev:0
  └─ ConstDecl('HURUF') → tab_index:36, type:char, lev:0
  └─ ConstDecl('PESAN') → tab_index:37, type:string, lev:0
  └─ TypeDecl('Hari') → tab_index:43, type:enum, lev:0
  └─ TypeDecl('Batas') → tab_index:44, type:subrange, lev:0
  └─ TypeDecl('Siswa') → tab_index:48, type:record, lev:0
  └─ TypeDecl('DataSiswa') → tab_index:49, type:array, lev:0
  └─ TypeDecl('Matriks') → tab_index:50, type:array, lev:0
  └─ VarDecl('i'), type:integer → tab_index:51, type:integer, lev:0, predefined
  └─ VarDecl('murid'), type:Siswa → tab_index:54, type:record, lev:0, predefined
  └─ VarDecl('kelas'), type:DataSiswa → tab_index:55, type:array, lev:0, predefined
  └─ VarDecl('papan'), type:Matriks → tab_index:56, type:array, lev:0, predefined
  └─ VarDecl('libur'), type:Hari → tab_index:57, type:enum, lev:0, predefined
└─ Block → block_index:0, lev:1
  └─ Assign('Var('i')' := UnaryOp(op: '+', operand: Num(10))) → type:void
  │   └─ 'i' → tab_index:51, type:integer, lev:0, predefined
  │   └─ UnaryOp '+' → type:integer
  │       └─ 10 → type:integer
  └─ Assign('Var('j')' := UnaryOp(op: '-', operand: Num(5))) → type:void
  │   └─ 'j' → tab_index:52, type:integer, lev:0, predefined
  │   └─ UnaryOp '-' → type:integer
  │       └─ 5 → type:integer
  └─ Assign('RecordAccess(base: Var('murid'), field: 'umur')' := Num(15)) → type:void
  │   └─ RecordAccess('umur') → type:integer
  │   └─ 15 → type:integer
  └─ Assign('RecordAccess(base: Var('murid'), field: 'nilai')' := BinOp(op: '*', left: Var('PI'), right: Num(2.0))) → type:void
  │   └─ RecordAccess('nilai') → type:real
  │   └─ BinOp '*' → type:real
  │       └─ 'PI' → tab_index:34, type:real, lev:0
  │       └─ 2.0 → type:real
  └─ Assign('RecordAccess(base: ArrayAccess(base: Var('kelas'), indices: [Num(1)]), field: 'lulus')' := Var('true')) → type:void
  │   └─ RecordAccess('lulus')
  │   └─ 'true' → tab_index:5, type:boolean, lev:0, predefined
  └─ Assign('ArrayAccess(base: ArrayAccess(base: Var('papan'), indices: [Num(1)]), indices: [Num(2)])' := Num(100)) → type:void
  │   └─ ArrayAccess → type:integer
  │   └─ 100 → type:integer
  └─ Assign('Var('libur')' := Var('Senin')) → type:void
  │   └─ 'libur' → tab_index:57, type:enum, lev:0, predefined
  │   └─ 'Senin' → tab_index:38, type:enum, lev:0
  └─ CaseStatement
  └─ ForLoop('i', to)
  │   └─ 1 → type:integer
  │   └─ 'MAX' → tab_index:35, type:integer, lev:0
  │   └─ Block → lev:1
  │       └─ IfStatement
  │           └─ BinOp '=' → type:boolean
  │           │   └─ 'i' → tab_index:51, type:integer, lev:0, predefined
  │           │   └─ 50 → type:integer
  │           └─ Assign('RecordAccess(base: ArrayAccess(base: Var('kelas'), indices: [Var('i')]), field: 'lulus')' := Var('true')) → type:void
  │               └─ RecordAccess('lulus')
  │               └─ 'true' → tab_index:5, type:boolean, lev:0, predefined
  │           └─ Assign('RecordAccess(base: ArrayAccess(base: Var('kelas'), indices: [Var('i')]), field: 'lulus')' := Var('false')) → type:void
  │               └─ RecordAccess('lulus')
  │               └─ 'false' → tab_index:6, type:boolean, lev:0, predefined

```

Semantic analysis completed

3.2 Test Case 2

Test Case berhasil karena pemanggilan procedure/function, tipe parameter, operasi boolean, operasi aritmatika, while, repeat, dan for sudah valid secara semantik.

INPUT

```
1  program testCase2;
2
3  var
4      x, y, z : integer;
5      status : boolean;
6      hasil : real;
7
8  procedure HitungMundur(start : integer; prefix : string);
9  var
10     counter : integer;
11 begin
12     counter := start;
13     while counter > 0 do
14     begin
15         counter := counter - 1;
16     end;
17 end;
18
19 function CekValid(a, b : integer; limit : real) : boolean;
20 var
21     temp : real;
22 begin
23     temp := (a + b) / 2.0;
24     if temp ≤ limit then
25         CekValid := true
26     else
27         CekValid := false;
28 end;
29
30 begin
31     x := 10;
32     y := 20;
33
34     HitungMundur(x, 'Mulai: ');
35
36     status := not CekValid(x, y, 15.5) and (x <> y) or (x = 10);
37
38     repeat
39         z := x * y div 2 mod 3;
40         x := x + 1;
41     until x ≥ 20;
42
43     for z := 10 downto 1 do
44     begin
45         y := y - 1;
46     end;
47 end.
```

OUTPUT

```

1 SEMANTIC ERROR REPORT
2 NO SEMANTIC ERRORS (Program is semantically valid)
3
4 TAB (hanya sebagian yang relevan):
5 idx  id          obj          type          ref  nrm  lev  adr  link
6 -----
7 33  testCase2     program    void          33   1   0   1   0
8 34  x             variable   integer        34   1   0   0   0
9 35  y             variable   integer        35   1   0   0   0
10 36  z             variable   integer        36   1   0   0   0
11 37  status        variable   boolean        37   1   0   0   0
12 38  hasil         variable   real           38   1   0   0   0
13 39  HitungMundur  procedure  void           39   1   0   0   0
14 40  start         parameter  integer        40   1   1   0   0
15 41  prefix        parameter  string         41   1   1   0   0
16 42  counter       variable   integer        42   1   1   0   0
17 43  CekValid      function   boolean        43   1   0   0   0
18 44  a             parameter  integer        44   1   1   0   0
19 45  b             parameter  integer        45   1   1   0   0
20 46  limit         parameter  real           46   1   1   0   0
21 47  temp          variable   real           47   1   1   0   0
22
23 btab:
24 idx  last  lpar  psze  vsze
25 -----
26 0    43    0     0     5
27 1    42    41     2     1
28 2    47    46     3     1
29
30 ATAB:
31 | (No arrays declared)
32

```

```

39 DECORATED AST:
40 ProgramNode('p5') → block_index:0, lev:0
41 └─ Declarations
42   └─ ConstDecl('c1') → tab_index:34, type:integer, lev:0
43   └─ ConstDecl('c2') → tab_index:35, type:integer, lev:0
44   └─ TypeDecl('t1') → tab_index:36, lev:0
45   └─ VarDecl('v1') → tab_index:37, type:array, lev:0
46   └─ FunctionDecl('f1') → block_index:1, tab_index:40, type:integer, lev:0, predefined
47     └─ ASTNode
48       └─ Block → block_index:1, lev:1
49         └─ IfStatement
50           └─ BinOp '≥' → type:boolean
51             └─ 'a1' → tab_index:41, type:integer, lev:1
52               └─ 10 → type:integer
53           └─ Assign('Var('a1')' := BinOp(op: '-', left: Var('a1'), right: Num(1))) → type:void
54             └─ 'a1' → tab_index:41, type:integer, lev:1
55             └─ BinOp '-' → type:integer
56               └─ 'a1' → tab_index:41, type:integer, lev:1
57                 └─ 1 → type:integer
58           └─ Assign('Var('a1')' := BinOp(op: '+', left: Var('a1'), right: Num(2))) → type:void
59             └─ 'a1' → tab_index:41, type:integer, lev:1
60             └─ BinOp '+' → type:integer
61               └─ 'a1' → tab_index:41, type:integer, lev:1
62                 └─ 2 → type:integer
63           └─ Assign('Var('f1')' := BinOp(op: 'mod', left: BinOp(op: 'div', left: Var('a1'), right: Num(2)), right: Num(3))) → type:void
64             └─ 'f1' → tab_index:40, type:integer, lev:0, predefined
65             └─ BinOp 'mod' → type:integer
66               └─ BinOp 'div' → type:integer
67                 └─ 'a1' → tab_index:41, type:integer, lev:1
68                   └─ 2 → type:integer
69               └─ 3 → type:integer
70         └─ ProcedureDecl('p1') → block_index:2, tab_index:42, type:void, lev:0, predefined
71           └─ Block → block_index:2, lev:1
72             └─ RepeatUntil
73       └─ Block → block_index:0, lev:1
74         └─ Assign('Var('v1')' := UnaryOp(op: '-', operand: Num(42))) → type:void
75           └─ 'v1' → tab_index:37, type:array, lev:0
76           └─ UnaryOp '-' → type:integer
77             └─ 42 → type:integer
78         └─ Assign('Var('v2')' := Num(3.1415)) → type:void
79           └─ 'v2' → tab_index:38, type:array, lev:0
80           └─ 3.1415 → type:real
81
82 Semantic analysis completed
83

```

```

115 DECOMPILED AST:
116 ProgramNode('testCase2') → block_index:0, lev:0
117 └─ Declarations
118   └─ VarDecl('x'), type:integer → tab_index:34, type:integer, lev:0, predefined
119   └─ VarDecl('status'), type:boolean → tab_index:37, type:boolean, lev:0, predefined
120   └─ VarDecl('result'), type:real → tab_index:38, type:real, lev:0, predefined
121   └─ ProcessDecl('stringIndex') → block_index:1, tab_index:39, type:VOID, lev:0, predefined
122   └─ ASTNode
123     └─ VarDecl('counter'), type:integer
124     └─ Block → block_index:1, lev:1
125       └─ Assign('Var('counter') := Var('start')') → type:VOID
126       └─ 'counter' → tab_index:42, type:integer, lev:1
127       └─ 'start' → tab_index:40, type:integer, lev:1
128       └─ nullLoop
129       └─ BinOp '>' → type:boolean
130       └─ 'counter' → tab_index:42, type:integer, lev:1
131       └─ 0 → type:integer
132       └─ Block → lev:1
133         └─ Assign('Var('counter') := BinOp(op: '-', left: Var('counter'), right: Num(1))) → type:VOID
134         └─ 'counter' → tab_index:42, type:integer, lev:1
135         └─ BinOp '-' → type:integer
136         └─ 'counter' → tab_index:42, type:integer, lev:1
137         └─ 1 → type:integer
138       └─ FunctionDecl('cekValid') → block_index:2, tab_index:43, type:boolean, lev:0, predefined
139       └─ ASTNode
140       └─ VarDecl('temp'), type:real
141       └─ Block → block_index:2, lev:1
142         └─ Assign('Var('temp') := BinOp(op: '/', left: BinOp(op: '+', left: Var('a'), right: Var('b')), right: Num(2.0))) → type:VOID
143         └─ 'temp' → tab_index:47, type:real, lev:1
144         └─ BinOp '/' → type:real
145         └─ BinOp '+' → type:integer
146         └─ 'a' → tab_index:48, type:integer, lev:1
147         └─ 'b' → tab_index:49, type:integer, lev:1
148         └─ 2.0 → type:real
149         └─ IfStatement
150         └─ BinOp '<' → type:boolean
151         └─ 'temp' → tab_index:47, type:real, lev:1
152         └─ 'limit' → tab_index:46, type:real, lev:1
153         └─ Assign('Var('cekValid') := Var('true')) → type:VOID
154         └─ 'cekValid' → tab_index:43, type:boolean, lev:0, predefined
155         └─ 'true' → tab_index:5, type:boolean, lev:0, predefined
156         └─ Assign('Var('cekValid') := Var('false')) → type:VOID
157         └─ 'cekValid' → tab_index:43, type:boolean, lev:0, predefined
158         └─ 'false' → tab_index:6, type:boolean, lev:0, predefined
159       └─ Block → block_index:0, lev:1
160         └─ Assign('Var('x') := Num(10)) → type:VOID
161         └─ 'x' → tab_index:34, type:integer, lev:0, predefined
162         └─ 10 → type:integer
163         └─ Assign('Var('y') := Num(20)) → type:VOID
164         └─ 'y' → tab_index:35, type:integer, lev:0, predefined
165         └─ 20 → type:integer
166         └─ BinOpModulo(...) → tab_index:39, type:VOID, lev:0
167         └─ 'x' → tab_index:34, type:integer, lev:0, predefined
168         └─ 'MulMod' → type:string
169         └─ Assign('Var('status') := BinOp(op: 'or', left: BinOp(op: 'and', left: UnaryOp(op: 'not', operand: ComplexExpression), right: BinOp(op: '<', left: Var('x'), right: Var('y'))), right: BinOp(op: '==', left: Var('x'), right: Num(10)))) → type:VOID
170         └─ 'status' → tab_index:37, type:boolean, lev:0, predefined
171         └─ BinOp 'or' → type:boolean
172         └─ BinOp 'and' → type:boolean
173         └─ UnaryOp 'not' → type:boolean
174         └─ cekValid(...) → tab_index:43, type:boolean, lev:0
175         └─ 'x' → tab_index:34, type:integer, lev:0, predefined
176         └─ 'y' → tab_index:35, type:integer, lev:0, predefined
177         └─ 25.5 → type:real
178         └─ BinOp '<' → type:boolean
179         └─ 'x' → tab_index:34, type:integer, lev:0, predefined
180         └─ 'y' → tab_index:35, type:integer, lev:0, predefined
181         └─ BinOp '==' → type:boolean
182         └─ 'x' → tab_index:34, type:integer, lev:0, predefined
183         └─ 10 → type:integer
184         └─ RepeatUntil
185         └─ ForLoop('2', count)
186         └─ 10 → type:integer
187         └─ 1 → type:integer
188         └─ Block → lev:1
189         └─ Assign('Var('y') := BinOp(op: '-', left: Var('y'), right: Num(1))) → type:VOID
190         └─ 'y' → tab_index:35, type:integer, lev:0, predefined
191         └─ BinOp '-' → type:integer
192         └─ 'y' → tab_index:35, type:integer, lev:0, predefined
193         └─ 1 → type:integer
194
195 Semantic analysis completed

```

3.3 Test Case 3

Test Case gagal karena v1, v2, dan v3 bertipe array, tetapi dipakai seperti nilai integer biasa: dibandingkan dengan </==, dijumlahkan/dikalikan, dijadikan iterator for, dan diisi langsung dengan nilai integer/real.

INPUT

```
test > milestone-3 > testCases.txt
1 { state skip: brace comment }
2 (*
3 state skip: paren-star comment
4 state skip: paren-star comment
5 state skip: paren-star comment
6 state skip: paren-star comment
7 state skip: paren-star comment
8 state skip: paren-star comment
9 *)
10
11 program p5;
12 const c1 = 123;
13 c2 = 123;
14 type t1 = hama;
15 var v1, v2, v3: array[1..3] of integer;
16
17 function f1(a1: integer): integer;
18 begin
19     if a1 ≥ 10 then
20         a1 := a1 - 1
21     else
22         a1 := a1 + 2;
23     f1 := a1 div 2 mod 3;
24 end;
25
26 procedure p1;
27 begin
28     repeat
29         while v1 < v2 do
30             begin
31                 v1 := v1 + 1
32             end;;
33         for v3 := 10 downto 1 do
34             begin
35                 v2 := v2 * 2 / 4;
36             end;
37         until v1 = v2;
38     end;
39
40     begin
41         v1 := -42;
42         v2 := 3.1415;
43     end.
44
```

OUTPUT

```

1 SEMANTIC ERROR REPORT
2 SEMANTIC ERRORS FOUND:
3 [SEMANTIC ERROR] Baris 29: Ekspresi biner tidak kompatibel untuk operator '<'
4 [SEMANTIC ERROR] Baris 31: Ekspresi biner tidak kompatibel untuk operator '+'
5 [SEMANTIC ERROR] Baris 35: Ekspresi biner tidak kompatibel untuk operator '*'
6 [SEMANTIC ERROR] Baris 33: Iterator harus bertipe integer atau subrange
7 [SEMANTIC ERROR] Baris 33: Tipe awal tidak kompatibel dengan iterator
8 [SEMANTIC ERROR] Baris 33: Tipe akhir tidak kompatibel dengan iterator
9 [SEMANTIC ERROR] Baris 37: Ekspresi biner tidak kompatibel untuk operator '='
10 [SEMANTIC ERROR] Baris 41: Tipe assignment tidak kompatibel
11 [SEMANTIC ERROR] Baris 42: Tipe assignment tidak kompatibel

```

12
13 TAB (hanya sebagian yang relevan):

14	idx	id	obj	type	ref	nrm	lev	adr	link
15	-----								
16	33	p5	program	void	33	1	0	11	0
17	34	c1	constant	integer	34	1	0	12	0
18	35	c2	constant	integer	35	1	0	13	0
19	36	t1	type	unknown	36	1	0	14	0
20	37	v1	variable	array	37	1	0	4	0
21	38	v2	variable	array	38	1	0	4	0
22	39	v3	variable	array	39	1	0	4	0
23	40	f1	function	integer	40	1	0	0	0
24	41	a1	parameter	integer	41	1	1	0	0
25	42	p1	procedure	void	42	1	0	0	0

26
27 btab:

28	idx	last	lpar	psze	vsze
29	-----				
30	0	42	0	0	3
31	1	41	41	1	0
32	2	0	0	0	0

33
34 ATAB:

35	arrays	xtyp	etyp	eref	low	high	elsz	size
36	-----							
37	1	0	0	0	1	3	4	12

38

```

39 DECORATED AST:
40 ProgramNode('p5') → block_index:0, lev:0
41   Declarations
42     ConstDecl('c1') → tab_index:34, type:integer, lev:0
43     ConstDecl('c2') → tab_index:35, type:integer, lev:0
44     TypeDecl('t1') → tab_index:36, lev:0
45     VarDecl('v1') → tab_index:37, type:array, lev:0
46     FunctionDecl('f1') → block_index:1, tab_index:40, type:integer, lev:0, predefined
47     ASTNode
48       Block → block_index:1, lev:1
49         IfStatement
50           BinOp '≥' → type:boolean
51             'a1' → tab_index:41, type:integer, lev:1
52             10 → type:integer
53           Assign('Var('a1')' := BinOp(op: '-', left: Var('a1'), right: Num(1))) → type:void
54             'a1' → tab_index:41, type:integer, lev:1
55             BinOp '-' → type:integer
56               'a1' → tab_index:41, type:integer, lev:1
57               1 → type:integer
58           Assign('Var('a1')' := BinOp(op: '+', left: Var('a1'), right: Num(2))) → type:void
59             'a1' → tab_index:41, type:integer, lev:1
60             BinOp '+' → type:integer
61               'a1' → tab_index:41, type:integer, lev:1
62               2 → type:integer
63           Assign('Var('f1')' := BinOp(op: 'mod', left: BinOp(op: 'div', left: Var('a1'), right: Num(2)), right: Num(3))) → type:void
64             'f1' → tab_index:40, type:integer, lev:0, predefined
65             BinOp 'mod' → type:integer
66               BinOp 'div' → type:integer
67                 'a1' → tab_index:41, type:integer, lev:1
68                 2 → type:integer
69               3 → type:integer
70       ProcedureDecl('p1') → block_index:2, tab_index:42, type:void, lev:0, predefined
71         Block → block_index:2, lev:1
72           RepeatUntil
73       Block → block_index:0, lev:1
74         Assign('Var('v1')' := UnaryOp(op: '-', operand: Num(42))) → type:void
75           'v1' → tab_index:37, type:array, lev:0
76           UnaryOp '-' → type:integer
77             42 → type:integer
78         Assign('Var('v2')' := Num(3.1415)) → type:void
79           'v2' → tab_index:38, type:array, lev:0
80           3.1415 → type:real
81
82 Semantic analysis completed
83

```

3.4 Test Case 4

Test Case gagal karena Point adalah type record, bukan subprogram/constructor, sehingga Point(0, 0) tidak valid. Selain itu, print belum dideklarasikan sebagai procedure.

INPUT


```
1  program Test4;
2
3  type Point =
4  record
5  |   x: integer;
6  |   y: integer
7  end;
8
9  var
10 |   now, ends: Point;
11 |   i: integer;
12
13 begin
14 |   now := Point(0, 0);
15 |   ends := Point(10, 10);
16 |   for i := 10 downto 0 do
17 |   begin
18 |       case i of
19 |       |   x : now.x := now.x + 1;
20 |       |   y : now.y := now.y + 1;
21 |       end
22 |   end;;
23 |   print('Point = ', x, 'And', y);
24 end.
25
```

OUTPUT

```

1 SEMANTIC ERROR REPORT
2 SEMANTIC ERRORS FOUND:
3 [SEMANTIC ERROR] Baris 14: Identifier 'Point' bukan subprogram
4 [SEMANTIC ERROR] Baris 14: Jumlah argumen tidak sesuai untuk pemanggilan 'Point'
5 [SEMANTIC ERROR] Baris 14: Tipe assignment tidak kompatibel
6 [SEMANTIC ERROR] Baris 15: Identifier 'Point' bukan subprogram
7 [SEMANTIC ERROR] Baris 15: Jumlah argumen tidak sesuai untuk pemanggilan 'Point'
8 [SEMANTIC ERROR] Baris 15: Tipe assignment tidak kompatibel
9 [SEMANTIC ERROR] Baris 23: Subprogram 'print' belum dideklarasikan
10
11 TAB (hanya sebagian yang relevan):
12 idx id obj type ref nrm lev adr link
13 -----
14 33 Test4 program void 33 1 0 1 0
15 34 x variable integer 34 1 0 0 0
16 35 y variable integer 35 1 0 0 0
17 36 Point type record 36 1 0 3 0
18 37 now variable record 37 1 0 0 0
19 38 ends variable record 38 1 0 0 0
20 39 i variable integer 39 1 0 0 0
21
22 btab:
23 idx last lpar psze vsze
24 -----
25 0 39 0 0 5
26
27 ATAB:
28 (No arrays declared)
29
30 DECORATED AST:
31 ProgramNode('Test4') → block_index:0, lev:0
32 | Declarations
33 | | TypeDecl('Point') → tab_index:36, type:record, lev:0
34 | | VarDecl('now'), type:Point → tab_index:37, type:record, lev:0, predefined
35 | | VarDecl('i'), type:integer → tab_index:39, type:integer, lev:0, predefined
36 | Block → block_index:0, lev:1
37 | | Assign('Var('now')' := ComplexExpression) → type:void
38 | | | 'now' → tab_index:37, type:record, lev:0, predefined
39 | | | ProcCall('Point') → tab_index:36, type:record, lev:0
40 | | | | 0 → type:integer
41 | | | | 0 → type:integer
42 | | Assign('Var('ends')' := ComplexExpression) → type:void
43 | | | 'ends' → tab_index:38, type:record, lev:0, predefined
44 | | | ProcCall('Point') → tab_index:36, type:record, lev:0
45 | | | | 10 → type:integer
46 | | | | 10 → type:integer
47 | | ForLoop('i', downto)
48 | | | 10 → type:integer
49 | | | 0 → type:integer
50 | | | Block → lev:1
51 | | | | CaseStatement
52 | | ProcCall('print')
53 | | | 'Point = ' → type:string
54 | | | 'x' → tab_index:34, type:integer, lev:0, predefined
55 | | | 'And' → type:string
56 | | | 'y' → tab_index:35, type:integer, lev:0, predefined
57
58 Semantic analysis completed
59

```

3.5 Test Case 5

Test Case berhasil karena array diakses melalui indeks terlebih dahulu, sehingga assignment dilakukan ke elemen array bertipe integer, bukan ke array secara langsung.

INPUT

```
1  { state skip: brace comment }
2  (*
3  state skip: paren-star comment
4  state skip: paren-star comment
5  *)
6
7  program testCase5;
8  const
9  |   c1 = 123;
10 |   c2 = 123;
11 type
12 |   hama = integer;
13 |   t1 = hama;
14 var
15 |   v1, v2, v3: array[1..3] of integer;
16 |   i: integer;
17
18 function f1(a1: integer): integer;
19 begin
20 |   if a1 ≥ 10 then
21 |       a1 := a1 - 1
22 |   else
23 |       a1 := a1 + 2;
24 |   f1 := a1 div 2 mod 3;
25 end;
26
27 procedure p1;
28 begin
29 |   repeat
30 |       while v1[1] < v2[1] do
31 |       begin
32 |           v1[1] := v1[1] + 1
33 |       end;;
34 |
35 |       for i := 10 downto 1 do
36 |       begin
37 |           v2[1] := v2[1] * 2 div 4;
38 |       end;
39 |   until v1[1] = v2[1];
40 end;
41
42 begin
43 |   v1[1] := -42;
44 |   v2[1] := 3;
45 end.
46
```

OUTPUT

1 SEMANTIC ERROR REPORT

2 NO SEMANTIC ERRORS (Program is semantically valid)

3

4 TAB (hanya sebagian yang relevan):

5	idx	id	obj	type	ref	nrm	lev	adr	link
6	-----								
7	33	testCase5	program	void	33	1	0	7	0
8	34	c1	constant	integer	34	1	0	9	0
9	35	c2	constant	integer	35	1	0	10	0
10	36	hama	type	integer	36	1	0	12	0
11	37	t1	type	integer	37	1	0	13	0
12	38	v1	variable	array	38	1	0	0	0
13	39	v2	variable	array	39	1	0	0	0
14	40	v3	variable	array	40	1	0	0	0
15	41	i	variable	integer	41	1	0	0	0
16	42	f1	function	integer	42	1	0	0	0
17	43	a1	parameter	integer	43	1	1	0	0
18	44	p1	procedure	void	44	1	0	0	0

19

20 btab:

21	idx	last	lpar	psize	vsze
22	-----				
23	0	44	0	0	4
24	1	43	43	1	0
25	2	0	0	0	0

26

27 ATAB:

28	arrays	xtyp	etyp	eref	low	high	elsz	size
29	-----							
30	1	0	0	0	1	3	4	12

31

```

32 DECORATED AST:
33 ProgramNode('testCase5') → block_index:0, lev:0
34   └─ Declarations
35     └─ ConstDecl('c1') → tab_index:34, type:integer, lev:0
36     └─ ConstDecl('c2') → tab_index:35, type:integer, lev:0
37     └─ TypeDecl('hama') → tab_index:36, type:integer, lev:0
38     └─ TypeDecl('t1') → tab_index:37, type:integer, lev:0
39     └─ VarDecl('v1') → tab_index:38, type:array, lev:0, predefined
40     └─ VarDecl('i'), type:integer → tab_index:41, type:integer, lev:0, predefined
41     └─ FunctionDecl('f1') → block_index:1, tab_index:42, type:integer, lev:0, predefined
42       └─ ASTNode
43         └─ Block → block_index:1, lev:1
44           └─ IfStatement
45             └─ BinOp '≥' → type:boolean
46               └─ 'a1' → tab_index:43, type:integer, lev:1
47               └─ 10 → type:integer
48             └─ Assign('Var('a1')' := BinOp(op: '-', left: Var('a1'), right: Num(1))) → type:void
49               └─ 'a1' → tab_index:43, type:integer, lev:1
50               └─ BinOp '-' → type:integer
51                 └─ 'a1' → tab_index:43, type:integer, lev:1
52                 └─ 1 → type:integer
53             └─ Assign('Var('a1')' := BinOp(op: '+', left: Var('a1'), right: Num(2))) → type:void
54               └─ 'a1' → tab_index:43, type:integer, lev:1
55               └─ BinOp '+' → type:integer
56                 └─ 'a1' → tab_index:43, type:integer, lev:1
57                 └─ 2 → type:integer
58             └─ Assign('Var('f1')' := BinOp(op: 'mod', left: BinOp(op: 'div', left: Var('a1'), right: Num(2)), right: Num(3))) → type:void
59               └─ 'f1' → tab_index:42, type:integer, lev:0, predefined
60               └─ BinOp 'mod' → type:integer
61                 └─ BinOp 'div' → type:integer
62                   └─ 'a1' → tab_index:43, type:integer, lev:1
63                   └─ 2 → type:integer
64                 └─ 3 → type:integer
65           └─ ProcedureDecl('p1') → block_index:2, tab_index:44, type:void, lev:0, predefined
66             └─ Block → block_index:2, lev:1
67               └─ RepeatUntil
68                 └─ Block → block_index:0, lev:1
69                   └─ Assign('ArrayAccess(base: Var('v1'), indices: [Num(1)])' := UnaryOp(op: '-', operand: Num(42))) → type:void
70                     └─ ArrayAccess → type:integer
71                     └─ UnaryOp '-' → type:integer
72                       └─ 42 → type:integer
73                   └─ Assign('ArrayAccess(base: Var('v2'), indices: [Num(1)])' := Num(3)) → type:void
74                     └─ ArrayAccess → type:integer
75                     └─ 3 → type:integer
76
77 Semantic analysis completed
78

```

BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Berdasarkan perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa *Semantic Analyzer* pada sistem ini berhasil dibangun menggunakan penelusuran yang didesain untuk menganalisis struktur program. Kompilasi bahasa Arion berhasil mengintegrasikan metode *Syntax-Directed Translation* (SDT) untuk mengonversi aturan produksi *grammar* menjadi *semantic action*. Dengan ini, program dapat mengeliminasi informasi sintaks yang sifatnya masih redundan dan akan dikonversi atau transformasi menjadi *Abstract Syntax Tree* (AST) yang lebih ringkas dan informatif.

1. *Semantic Analyzer* telah mencakup berbagai validasi aturan deklarasi bahasa Arion, seperti pemeriksaan keunikan *identifier* dan pendeteksian deklarasi ganda pada konstanta, tipe data, variabel, hingga parameter subprogram.
2. *Semantic Analyzer* berhasil mendukung aturan kompatibilitas tipe (*type checking*) pada berbagai *statement* dasar, seperti *assignment*, *if*, *case*, perulangan (*while*, *repeat*, dan *for*), serta kesesuaian argumen pada saat pemanggilan prosedur atau fungsi.
3. Selain itu, implementasi *Semantic Analyzer* juga didukung oleh pengelolaan tabel simbol seperti **tab**, **btab**, dan **atab** untuk merepresentasikan *identifier*, struktur blok, serta informasi array yang sudah terurut, sehingga proses validasi semantik dapat dilakukan dengan lebih konsisten.

Secara keseluruhan, hasil implementasi menunjukkan bahwa *semantic analyzer* telah berhasil melakukan analisis semantik sesuai bahasa Arion yang dikembangkan serta menghasilkan representasi AST yang valid untuk mendukung proses generasi kode nantinya.

4.2. Saran

Untuk pengembangan lebih lanjut, proses pembentukan AST dan analisis semantik pada compiler Arion masih dapat ditingkatkan agar hasil analisis menjadi lebih akurat dan mudah dikembangkan. Pada bagian *semantic analysis*, sistem dapat dikembangkan agar mampu menangani error dengan lebih baik sehingga satu kesalahan tidak memunculkan banyak error tambahan yang membingungkan. Selain itu, aturan semantik juga dapat diperluas, seperti pengecekan batas array, validasi tipe data yang lebih detail, serta pengecekan jalur eksekusi program untuk mendeteksi kode yang tidak akan dijalankan atau fungsi yang belum memiliki nilai pengembalian yang sesuai.

BAB V

LAMPIRAN

Link Repository Github: <https://github.com/staplesmaster/SSS-Tubes-IF2224-2026>

Pembagian Tugas:

Nama	NIM	Bagian	Persentase
Geraldo Artemius	13524005	• Semantic Analyzer • Laporan	20%
Mikhael Andrian Yonatan	13524051	• AST Node & Visitor • AST Converter • Tester & Debugger	20%
Junior Natra Situmorang	13524055	• AST Node • Integration • Writer • Error Reporter	20%
Reynard Nathanael	13524103	• Symbol Info • Type Rules • Laporan	20%
Nicholas Luis Chandra	13524105	• Symbol Table • Laporan	20%

BAB VI

REFERENSI

- [1] GeeksforGeeks, “Syntax Directed Translation in Compiler Design,” *GeeksforGeeks*, Jan. 05, 2017. Available: <https://www.geeksforgeeks.org/compiler-design/syntax-directed-translation-in-compiler-design/>
- [2] GeeksforGeeks, “Abstract Syntax Tree vs Parse Tree,” *GeeksforGeeks*, Jan. 12, 2023. Available: <https://www.geeksforgeeks.org/compiler-design/abstract-syntax-tree-vs-parse-tree/>
- [3] “Attributed Grammars in Compiler Design,” *Tutorialspoint.com*, 2025. Available: https://www.tutorialspoint.com/compiler_design/compiler_design_attributed_grammars.htm
- [4] GeeksforGeeks, “S Attributed and L Attributed SDTs in Syntax Directed Translation,” *GeeksforGeeks*, Jul. 17, 2018. Available: <https://www.geeksforgeeks.org/compiler-design/s-attributed-and-l-attributed-sdts-in-syntax-directed-translation/>
- [5] “GeeksforGeeks,” *GeeksforGeeks*, Apr. 24, 2018. Available: <https://www.geeksforgeeks.org/compiler-design/symbol-table-compiler/>